Guide Power BI — Analyse du secteur bancaire tunisien

Chaque transformation Power Query et chaque mesure DAX, expliquées étape par étape

Ahmed Ben Arfa · version du 23/10/2026 (générée à partir de docs/guide_powerbi.md, powerquery/*.pq, dax/*.dax et docs/feuille_de_route.md)

Ce guide explique, étape par étape, chaque requête Power Query et chaque mesure DAX du rapport : ce qu'elle calcule et pourquoi. Il est aussi publié en PDF, régénéré après chaque requête ou mesure : `docs/Guide_PowerBI.pdf` (`python scripts/generer_pdf.py guide`), avec en annexe le code M exact de chaque requête et la feuille de route (décisions). Il se lit à côté du projet Power BI (`powerbi/`) et des fichiers `powerquery/*.pq` (le texte exact de chaque requête). La spécification complète est dans `docs/SPECIFICATION.md`.

Principe : Python ne fait que lire les PDF (`collecte_banques.py`, `scripts/`). Tout ce qui peut se faire dans Power Query s'y fait, pour que la préparation des données reste visible dans le rapport.

1. Créer et enregistrer le projet `.pbip`

1. Power BI Desktop : **Fichier › Options et paramètres › Options › Fonctionnalités en préversion**, cocher **Enregistrement du projet Power BI (.pbip)** et **Stocker le modèle sémantique au format TMDL**, puis redémarrer Power BI Desktop (si ces cases n'existent plus, la fonction est déjà activée par défaut).
2. **Fichier › Options › Paramètres régionaux** (fichier actif) : laisser la langue du rapport en français ; les requêtes fixent elles-mêmes la culture `en-US` pour lire les nombres à point décimal.
3. Rapport vide : **Fichier › Enregistrer sous**, type **Fichiers de projet Power BI (*.pbip)**, dossier `D:\Projet\Analyse-secteur-bancaire-tunisien\powerbi\`, nom `SecteurBancaireTunisien`. Attention : le type par défaut est `.pbix` (un seul fichier binaire, illisible hors de Power BI) ; seul le type `.pbip` produit les fichiers texte du modèle que l'on peut lire et versionner.
4. Power BI crée `SecteurBancaireTunisien.pbip`, le dossier `SecteurBancaireTunisien.Report\` (pages et visuels) et le dossier `SecteurBancaireTunisien.SemanticModel\` (modèle : requêtes, tables, relations, mesures, au format texte TMDL dans `definition\`). Power BI n'a pas créé de `.gitignore` dans ce projet : c'est celui de la racine du dépôt qui exclut le cache des données (`cache.abf`) et les réglages locaux ; seules les définitions sont versionnées, jamais les données.

Où sont les requêtes dans le projet. Le modèle est stocké en texte (TMDL) dans `powerbi/SecteurBancaireTunisien.SemanticModel/definition/` :

Fichier	Contenu
`expressions.tmdl`	le paramètre `CheminDonnees` et les requêtes intermédiaires non chargées (`Fichiers_Donnees`, `Versions`)
`tables/<Table>.tmdl`	une table chargée (`Dim_Indicateur`, `Fait_Indicateurs` …) : ses colonnes (type, synthèse, colonne de tri) et sa requête (partition `m`)
`model.tmdl`	options du modèle, ordre des requêtes dans Power Query (`PBI_QueryOrder`), liste des tables

Le code M de chaque requête est le même que celui du fichier `powerquery/<Requête>.pq`, gardé comme copie lisible : une modification faite dans Power Query doit être reportée dans le `.pq` (et inversement). Les requêtes sont écrites directement dans ces fichiers, Power BI Desktop fermé ; à la réouverture, Power BI les lit et les affiche dans Power Query.

Niveaux de confidentialité. À la première actualisation, Power BI demande le niveau de chaque source (dossier du projet, fichier Excel, adresse GitHub) : choisir le dossier du projet (pas tout le disque) et le même niveau pour

toutes, **Professionnel** (« Organisationnel » dans certaines versions), sinon les fusions entre sources peuvent être bloquées. Pour l'adresse GitHub, accès **Anonyme** (dépôt public). Ces réglages sont enregistrés sur le poste, pas dans le projet : une personne qui clone le dépôt les donne une fois.

Aperçu annulé. Power Query ne calcule l'aperçu que d'une requête à la fois : en passant d'une requête à l'autre, ou pendant un chargement, l'aperçu en cours est annulé (« L'actualisation d'aperçu a été annulée… aucun impact sur vos données chargées ») ; **Réessayer** ou **Accueil › Actualiser l'aperçu** le relance. Power BI enregistre alors `PBI_ResultType = Exception` pour cette requête dans le TMDL (icône « ? » dans Power Query), sans effet sur les données chargées ; c'est fréquent pour `Banques_Actionnariat` (lecture d'un fichier Excel). Pour le limiter : **Fichier › Options et paramètres › Options › Fichier actif › Chargement des données**, cocher « Autoriser le téléchargement des aperçus de données en arrière-plan ».

Ne sont pas versionnés (`.gitignore`) : `.pbi/localSettings.json` (réglages et signature de sécurité propres au poste) et l'ancien `SecteurBancaireTunisien.pbix` . **Le cache des données (`.pbi/cache.abf`) est versionné** (décision du 23/10/2026) : le projet s'ouvre avec ses données sans actualiser, donc sans changer `CheminDonnees` ni répondre aux questions d'accès. Toutes ces données sont déjà publiques dans le dépôt. Le fichier est compressé : l'audit ne peut pas lire son contenu. Il est commité seulement après une vérification dans Power BI Desktop suivie d'un enregistrement, pour que les données publiées correspondent au modèle.

Vérifier après une réouverture. Ouvrir `SecteurBancaireTunisien.pbip` , **Accueil › Transformer les données** : les requêtes apparaissent dans l'ordre de la section 2, les intermédiaires en italique (chargement désactivé). Revenir au rapport et **Actualiser**. Contrôles (calculés sur les CSV) :

Table	Contrôle	Attendu
`Dim_Indicateur`	nombre de lignes	27 (24 indicateurs trimestriels + 3 des états financiers)
`Fait_Indicateurs`	nombre de lignes	7 173 (7 137 avant la saisie des composantes BTE 2020–2021)
`Fait_Indicateurs`	AB 2026T3, PNB	`Valeur_Trim` = 150,021 ; `Valeur_Cumul` = 512,448 (relevés sur l'image, `tests/valeurs_reference.csv`)
`Fait_Indicateurs`	AB 2020T1, CREDITS (stock)	`Valeur_Trim` , `Source_Trim` , `Fichier_Trim` vides ; `Valeur_Cumul` = 5 670,677
`Fait_Indicateurs`	BTE 2021T4, INT_PERCUS	`Valeur_Trim` = 22,367, `Source_Trim` = `revisee_T4_N+1` , `Fichier_Trim` = `BTE_2022T4.pdf` (22,069 publié à l'origine)
`Fait_Indicateurs`	BTE 2020T2, COMM_CHG	`Valeur_Trim` = 0,005 (5 515 publiés à l'origine, révisés dans BTE 2021T2)
`Fait_Indicateurs`	`Rupture_Cumul`	6 723 « non », 450 « oui », aucune case vide
`Revisions`	nombre de lignes	14 884 (avec `Fait_Indicateurs` : les 26 429 lignes de `indicateurs_versions.csv`)
`Revisions`	BH 2024T4, PNB, cumul, version `initiale`	`valeur_mdt` = 765,932 ; `valeur_retenue` = 701,188 ; `Ecart_MDT` = −64,744 ; « révision importante (plus de 2 %) »
`Revisions`	`Type_Revision`	7 639 retraitements mineurs, 5 677 identiques, 1 133 révisions importantes, 241 « 31/12 provisoire écarté », 99 reclassements, 70 documents en image, 25 colonnes fausses
`Banques_Actionnariat`	nombre de lignes	15 (12 banques ; 2 périodes pour BTE, UBCI et Wifak) ; aucune erreur dans `actionnariat`
`Dim_Banque`	nombre de lignes	12
`Dim_Banque`	UBCI	`actionnaire_reference` = Groupe Carte ; `part_capital_pct` = 39,86 ; `actionnariat` = privé sans contrôle étranger

Table	Contrôle	Attendu
`Dim_Banque`	BTE	Abou Dhabi Investment Authority (ADIA) ; 40,80 ; mixte ; `date_actionnariat` = 31/12/2024
`Dim_Banque`	WIB	`ticker_bvmt` = WIFAK ; groupe = Groupe Ben Ayed
`Operations_Titres` (Power Query, requête intermédiaire)	nombre de lignes (barre d'état en bas)	63
`Operations_Titres`	BIAT 04/05/2021 (division du nominal)	`facteur` = 0,5
`Operations_Titres`	BIAT 10/05/2019	`facteur_cumule_prix` = 0,416667 (produit des 3 opérations sur le capital qui suivent)
`Operations_Titres`	BT 10/05/2022 (attribution + dividende)	`facteur` = 0,796446 ; `facteur_prix` = 0,834179
`Evenements`	nombre de lignes	11
`Evenements`	BH_RETRAITEMENT_2024	`date` = 31/12/2024 ; `montant_mdt` = −64,744
`Fait_Cours`	nombre de lignes	23 460 (1 955 séances × 12 banques)
`Fait_Cours`	BIAT 02/01/2019	`cloture` = 118 ; `facteur_prix` = 0,416667 ; `cloture_ajustee_prix` = 49,1667 ; `facteur_rendement` = 0,272016 ; `rendement` vide (première séance)
`Fait_Cours`	BIAT 31/12/2025	`cloture` = 111 ; `facteur_prix` = 1 ; `facteur_rendement` = 0,9625 (dividende de mai 2026)
`Fait_Cours`	BT 10/05/2022 (jour de l'attribution)	`cloture` = 5,05 ; `facteur_prix` = 1 ; `facteur_rendement` = 0,795311
`Fait_Cours`	`sans_transaction` = Vrai	3 074 séances (UBCI 52,7 %, Wifak 42,6 %, BTE 37,6 % de leurs séances)
`Variations_Comparables`	nombre de lignes	10 981 (11 285 du fichier moins 304 lignes « trimestre » de stocks)
`Variations_Comparables`	BH 2025T4, PNB	`cumul` : 681,479 contre 701,188, `Variation_Pct` = −2,8 % ; `trim` : 189,758 contre 177,219, +7,1 %
`Fait_Etats_Financiers`	nombre de lignes	504 (12 banques × 14 dates × 3 codes) ; `facteur_annualisation` = 2 sur 252 lignes (semestres), 1 sur 252 (exercices)
`Fait_Etats_Financiers`	BNA 30/06/2025, RES_NET	155,069 ; `version` = initiale ; statut « revisee non retenue (modification comptable incluse) »
`Fait_Etats_Financiers`	UIB 30/06/2020, RES_NET	17,912 (révisée, moratoire Covid ; −41,182 publié à l'origine)
`Fait_Etats_Financiers`	BIAT 31/12/2024, CAP_PROPRES_EF	2 226,391
`Revisions_Etats_Financiers`	nombre de lignes	423 (avec `Fait_Etats_Financiers` : les 927 lignes du fichier)
`Revisions_Etats_Financiers`	UIB 30/06/2020, RES_NET, version `initiale`	−41,182 publié ; 17,912 retenu ; `Ecart_MDT` = 59,094 ; « révision importante (plus de 2 %) »
`Revisions_Etats_Financiers`	BNA 30/06/2025, RES_NET, version `revisee`	137,318 ; 155,069 retenu ; « révisée non retenue (modification comptable incluse) »
`Revisions_Etats_Financiers`	`Type_Revision`	390 identiques, 24 retraitements mineurs, 8 révisions importantes, 1 révisée non retenue

Table	Contrôle	Attendu
`Controle`	nombre de lignes	336 (12 banques × 28 trimestres) ; somme de `nb_alertes` = 968
`Controle`	`Categorie_Statut`	91 conformes, 228 écarts signalés, 13 saisis à la main, 2 repris du document suivant, 2 alertes bloquantes (UBCI 2025T3, UIB 2023T2)
`Controle`	BTE 2021T4	`statut` = saisie manuelle ; `date_arrete` = 31/12/2021 ; `nb_alertes` = 7
`Ecarts_Annuels` (Power Query, intermédiaire)	nombre de lignes	22
`Controle_Annuel`	nombre de lignes	288 (12 banques × 6 années × 4 indicateurs)
`Controle_Annuel`	`Statut_Clair`	205 « somme des trimestres = cumul annuel », 41 « en attente du document de l'année suivante », 20 « changement de méthode en cours d'année », 22 « écart expliqué : … » (11 retraitements de la banque, 7 erreurs dans le document de la banque, 3 reclassements, 1 arrondi), **0** « écart sans cause écrite »
`Controle_Annuel`	ATB 2021, CHG_EXPL	somme 309,5 ; cumul 312,374 ; `Ecart_MDT` = −2,874 ; « écart expliqué : retraitement de la banque » ; preuve ATB_2022T2.pdf p. 1
`Controle_Annuel`, `Revisions`, `Revisions_Etats_Financiers`	`Ecart_MDT`	affiché avec trois décimales (format `#,0.000`) : 0,001 et non 0,000999999… (imprécision de calcul des nombres décimaux, valeur inchangée)
`Gabarits` (Power Query, intermédiaire)	nombre de lignes	46
`Definitions_Charges`	nombre de lignes	13 (une par banque, deux pour Attijari)
`Definitions_Charges`	TJARI	« sans dotations » de 2019T4 à 2021T3 ; « avec reste non detaille » de 2021T4 à 2026T3
`Definitions_Capitaux_Propres`	nombre de lignes	12 ; « resultat inclus » pour ATB et BTE ; « hors resultat, apres affectation au 31/12 » pour AB et BIAT
`Fait_Indicateurs`, `Variations_Comparables`, `Fait_Etats_Financiers`	`Segment_Actionnariat`	jamais vide ; nombres de lignes inchangés (7 173, 10 981, 504)
`Fait_Indicateurs`	UBCI 2020T4 / 2021T1	« groupe étranger » / « privé sans contrôle étranger » (cession du 16/03/2021)
`Fait_Indicateurs`	BTE 2024T3	« mixte » (seconde période, à partir du 01/07/2024)

Ajout manuel d'une requête (sans passer par les fichiers TMDL) : **Accueil › Transformer les données**, puis **Nouvelle source › Requête vide**, **Éditeur avancé**, coller le contenu du fichier `.pq`, renommer la requête exactement comme le fichier (sans `.pq`). Requête intermédiaire : clic droit › décocher **Activer le chargement**. Puis **Fermer et appliquer** et enregistrer (Ctrl+S).

2. Requêtes Power Query

Ordre de création : chaque requête n'utilise que des requêtes déjà créées.

#	Requête	Chargée	Source	Rôle
1	`CheminDonnees`	paramètre	—	dossier du projet sur ce poste

#	Requête	Chargée	Source	Rôle
1b	`AdresseDepot`	paramètre	—	adresse des fichiers bruts du dépôt GitHub
2	`Fichiers_Donnees`	non	connecteur Dossier	liste des CSV de `donnees_publiees\`
3	`Versions`	non	`indicateurs_versions.csv`	toutes les versions des indicateurs, typées
4	`Dim_Indicateur`	oui	`dim_indicateur.csv` + 3 lignes ajoutées	libellé, catégorie, nature (flux / stock), ordre
5	`Fait_Indicateurs`	oui	`Versions` + fusion avec `Dim_Indicateur`	valeur retenue, trimestre seul et cumul côte à côte
6	`Revisions`	oui	`Versions`	versions non retenues et écart à la valeur retenue (page qualité)
7	`Banques_Actionnariat`	oui	`donnees_reference anques.xlsx` (connecteur Excel)	actionnariat par période, avec source et page
8	`Dim_Banque`	oui	`dim_banque.csv` + fusion avec `Banques_Actionnariat`	code, nom, ticker, actionnariat actuel
8b	`Actionnariat_A_La_Date`	fonction	`Banques_Actionnariat`	catégorie d'actionnariat d'une banque à une date (segment des faits)
9	`Operations_Titres`	non	`operations_titres.csv` sur GitHub (source URL)	facteurs d'ajustement des cours
10	`Evenements`	oui	`evenements.csv` sur GitHub (source URL)	événements qui expliquent une rupture de série
11	`Fait_Cours`	oui	`cours_banques.csv` + fusion avec `Operations_Titres`	cours quotidiens, cours ajustés, rendement, séances sans transaction
12	`Variations_Comparables`	oui	`variations_comparables.csv` + fusion avec `Dim_Indicateur`	valeur N et N-1 du même document, variation sur un an
13	`Fait_Etats_Financiers`	oui	`etats_financiers_versions.csv`	résultat net, total de l'actif, capitaux propres du bilan (ROE, ROA)
14	`Revisions_Etats_Financiers`	oui	`etats_financiers_versions.csv`	versions écartées des états financiers (page qualité)
15	`Controle`	oui	`controle_extraction.csv`	statut d'extraction de chaque document (page qualité)
16	`Ecarts_Annuels`	non	`docs\ecarts_annuels.csv` (connecteur Fichier)	cause de chaque écart du contrôle annuel
17	`Controle_Annuel`	oui	`controle_annuel.csv` + fusion avec `Ecarts_Annuels`	somme des trimestres contre cumul annuel, statut et cause
18	`Gabarits`	non	`gabarits.csv` (connecteur Fichier)	définitions de chaque banque, par mise en page
19	`Definitions_Charges`	oui	`Gabarits` (regroupement)	contenu des charges opératoires publiées, par période
20	`Definitions_Capitaux_Propres`	oui	`Gabarits` (lignes distinctes)	contenu des capitaux propres trimestriels publiés

2.1 `CheminDonnees` (paramètre)

Texte : le dossier du projet (`D:\Projet\Analyse-secteur-bancaire-tunisien`), qui contient `donnees_publiees\` (CSV finaux, versionnés), `donnees_reference\` (fichier Excel des banques) et `docs\`. **Pourquoi** : un seul endroit à changer si le projet est copié sur un autre poste (**Accueil › Gérer les paramètres**) ; aucune requête n'écrit de chemin en dur. Après un clonage du dépôt, il suffit de donner à `CheminDonnees` le dossier du clone puis d'actualiser : toutes les données lues sont dans le dépôt.

2.2 `Fichiers_Donnees` (intermédiaire, non chargée)

Étape	Ce qu'elle fait
`Source`	`Folder.Files` : tous les fichiers de `donnees_publiees\` (connecteur Dossier)
`CsvDuDossier`	garde les `.csv` posés directement dans ce dossier (pas `LISEZMOI.md`)
`Colonnes`	garde le nom, le contenu binaire et la date de modification
`Renommees`	noms en français : `fichier`, `contenu`, `modifie_le`

Pourquoi : les requêtes suivantes prennent leur fichier par son nom dans cette table (`Fichiers_Donnees{[fichier = "…"]}[contenu]`) ; la date de modification permet de vérifier, sur la page qualité, que les CSV sont ceux de la dernière publication. **Pourquoi `donnees_publiees\` et pas `donnees\`** : `donnees\` est le dossier de travail des scripts (PDF, archives, caches), non versionné ; `donnees_publiees\` contient seulement les CSV finaux, sans chemin local, versionnés avec le dépôt. Il est régénéré par `python scripts/publier_donnees.py` (voir `donnees_publiees/LISEZMOI.md`).

2.3 `Versions` (intermédiaire, non chargée)

Une ligne par banque × trimestre × indicateur × mesure (`trim` = trimestre seul, `cumul` = depuis janvier ou solde) × version (`initiale` = publiée dans le document du trimestre, `revisee_…` = colonne N-1 du document suivant, `revisee_31-12_…` = 31/12 provisoire). 26 223 lignes au 22/10/2026.

Étape	Ce qu'elle fait
`Fichier`	prend `indicateurs_versions.csv` dans `Fichiers_Donnees`
`Source`	lit le CSV : séparateur `;`, encodage UTF-8 (65001, le BOM est reconnu)
`EnTetes`	première ligne = noms de colonnes
`Types`	types de données **avec les paramètres régionaux `en-US`** : le CSV écrit `29.585` ; lu en français, ce serait 29 585
`VidesEnNull`	une case de texte vide devient `null` (« pas de note », « pas de rupture »)

Pourquoi intermédiaire : ce fichier mélange la valeur retenue et les versions écartées. Deux requêtes chargées en partent avec un filtre explicite : `Fait_Indicateurs` (version retenue seulement) et `Revisions` (versions non retenues, page qualité). Une valeur manquante reste `null`, jamais zéro.

2.4 `Dim_Indicateur` (chargée)

Une ligne par code d'indicateur (27 lignes).

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`	lit `dim_indicateur.csv` dans `Fichiers_Donnees`
`Types`	types de données (paramètres régionaux `en-US`) ; `ordre` en entier
`SourceIndicateurs`	**colonne ajoutée** `source` = « indicateurs trimestriels »

Étape	Ce qu'elle fait
`EtatsFinanciers`	table de 3 lignes écrite dans la requête : `RES_NET` (flux), `TOTAL_ACTIF` et `CAP_PROPRES_EF` (stocks), source « états financiers »
`Ajoutees`	**ajout** des 3 lignes à la suite des 24 codes

Pourquoi ajouter les codes des états financiers : un événement peut porter sur le résultat net (ex. moratoire Covid de l'UIB au 30/06/2020) ; avec son code dans la dimension, la relation garde l'annotation sur la page « Rentabilité » (spécification 6). La colonne `libelle` est triée par `ordre` dans le modèle (ordre d'affichage des indicateurs, pas l'ordre alphabétique).

2.5 `Fait_Indicateurs` (chargée)

Table de faits des indicateurs trimestriels : une ligne par banque × trimestre × indicateur (7 173 lignes au 23/10/2026), avec le trimestre seul et le cumul côte à côte.

Étape	Ce qu'elle fait
`Source`	part de `Versions` (référence, pas de nouvelle lecture du fichier)
`VersionRetenue`	**filtre sur la version retenue** : `version = source_valeur`, une seule version par banque × trimestre × indicateur × mesure (11 545 lignes)
`Colonnes`	garde les clés, la mesure (`trim` / `cumul`), la valeur retenue, la version, le fichier, la page et la rupture de série
`Fusion` , `Nature`	**fusion avec `Dim_Indicateur`** sur `code_indicateur` (jointure externe gauche) : nature de l'indicateur
`Retenir` , `SansTrimDesStocks`	**colonne conditionnelle** : faux pour la mesure `trim` d'un stock, puis filtre ; écarte 320 lignes (au T1, copie du solde chez AB, BTE, STB, Attijari, UBCI ; STB 2019T4 et UBCI 2024–2026 : mouvement imprimé dans la colonne du trimestre, qui n'est pas un solde)
`RuptureNon`	« non » au lieu d'une case vide dans `rupture_serie` (le détail de la rupture reste vide)
`SansColonnesAjoutees`	retire `nature` et `retenir`
`Renommees`	noms courts des attributs (`Valeur` , `Source` , `Fichier` , `Page` , `Rupture` , `Detail_Rupture`)
`Depivotees`	**dépivotage** : une ligne par attribut ; une case vide (pas de rupture) ne donne pas de ligne
`NomColonne`	colonne conditionnelle du nom final : attribut + mesure (`Valeur_Trim` , `Source_Cumul` …)
`SansMesure`	retire `mesure` et `attribut` , devenus inutiles
`Pivotees`	**pivot** sans agrégation sur cette liste fixe de 12 colonnes (une seule valeur par case : une erreur ici signalerait un doublon dans les versions)
`Types`	nombres pour les valeurs, entiers pour les pages, texte pour le reste
`Periodes` , `Segment`	**segment d'actionnariat** : périodes de `Banques_Actionnariat` gardées en mémoire, puis colonne `Segment_Actionnariat` = fonction `Actionnariat_A_La_Date` (catégorie de la banque à la date d'arrêté, section 2.12h)

Pourquoi un filtre explicite : la règle des versions (spécification 5.2) se voit dans Power Query : les niveaux affichés sont toujours la version révisée quand elle existe ; les versions écartées restent consultables dans la requête `Revisions` . **Pourquoi trimestre et cumul côte à côte** : les mesures DAX choisissent la colonne (trimestre seul pour l'analyse trimestrielle, cumul pour l'année et pour les soldes) sans filtre sur une colonne `mesure` . `Valeur_Cumul` est toujours rempli ; `Valeur_Trim` est vide pour les soldes (crédits, dépôts, capitaux propres : la valeur à la date est dans `Valeur_Cumul`) et pour un flux dont le trimestre seul n'est publié nulle part (BTE : autres produits d'exploitation à quelques dates) : jamais remplacé par zéro. `Fichier_*` et `Page_*` gardent la source de chaque valeur ; `Rupture_*` (« oui ») et `Detail_Rupture_*` alimentent les marqueurs et les info-bulles des graphiques.

2.6 `Revisions` (chargée)

Toutes les versions qui ne sont pas retenues : une ligne par banque × trimestre × indicateur × mesure × version écartée (14 884 lignes au 23/10/2026). Page « qualité des données ».

Étape	Ce qu'elle fait
`Source`	part de `Versions` (référence)
`VersionsNonRetenues`	**filtre inverse** de `Fait_Indicateurs` : `version <> source_valeur` ; les deux tables ensemble redonnent toutes les lignes du fichier
`Colonnes`	clés, version, valeur publiée (`valeur_mdt`), fichier et page de cette version, valeur et version retenues, statut, note, mention de retraitement
`EcartMDT`	**colonne calculée** `Ecart_MDT` = valeur retenue − valeur de cette version (vide si l'une manque, jamais zéro) ; calculée ici plutôt que reprise du CSV
`EcartPct`	`Ecart_MDT` rapporté à la valeur de la version (vide si elle est nulle ou absente)
`OrigineVersion`	**colonne conditionnelle** : document du trimestre, 31/12 provisoire (document T1 à T3 suivant) ou colonne N-1 du document suivant
`TypeRevision`	**colonne conditionnelle** : statut en clair (identique à l'arrondi près, retraitement mineur, révision importante, reclassement, document en image, 31/12 provisoire écarté pour les capitaux propres, colonne fausse écartée)

Pourquoi : chaque valeur affichée dans le rapport est la version retenue ; cette table montre ce que la banque avait publié avant, et de combien elle a révisé (ex. PNB 2024 de BH : 765,932 publié dans BH_2024T4, 701,188 retenu, « retraité pour des raisons de comparabilité »). Les 3 lignes sans valeur retenue sont les capitaux propres BTE au 31/12 provisoire (2021 et 2022), écartés par la règle.

2.7 `Banques_Actionnariat` (chargée)

Actionnariat des 12 banques lu dans le fichier Excel `donnees_reference▯anques.xlsx` (table Excel nommée `Banques`, 15 lignes) : une ligne par banque et par période d'actionnaire de référence.

Étape	Ce qu'elle fait
`Classeur`	**connecteur Excel** : `Excel.Workbook` sur le fichier, première ligne = en-têtes
`Banques`	prend la **table Excel** `Banques` (pas la feuille) : ses limites suivent les lignes ajoutées dans Excel
`Types`	texte, nombre (`part_capital_pct`, en % du capital), dates (`date_reference`, `date_debut`, `date_fin`)
`CategorieValide`	une catégorie hors des quatre de la règle (public, groupe étranger, mixte, privé sans contrôle étranger) devient une **erreur visible** à l'actualisation

Pourquoi un fichier Excel : l'actionnariat ne vient pas des PDF d'indicateurs ; il est relevé à la main dans des sources officielles (fiche émetteur de la Bourse, états financiers, rapports annuels), avec la source et la page de chaque ligne, dans un fichier qu'on peut relire et corriger dans Excel (`python scripts/ecrire_banques_xlsx.py` le régénère). Règle de classement par le contrôle : spécification 5.3. Les périodes (`date_debut`, `date_fin`) servent au segment d'actionnariat des faits : UBCI change de catégorie le 16/03/2021 (BNP Paribas → groupe Carte) ; Wifak et BTE changent d'actionnaire de référence sans changer de catégorie. Pour BTE, la date de l'augmentation de capital du second semestre 2024 n'est pas connue : la seconde période commence au 01/07/2024 (« date exacte inconnue, entre le 30/06 et le 31/12/2024 »), pour que les périodes se suivent sans trou (contrôlé par `tests/test_actionnariat.py`).

2.8 `Dim_Banque` (chargée)

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`, `Types`	lit `dim_banque.csv` (code, nom, ticker de la Bourse)
`SansColonnesVides`	retire `actionnariat` et `groupe`, vides dans le CSV
`ActionnariatActuel`	filtre de `Banques_Actionnariat` : la ligne sans `date_fin` (une par banque)
`Fusion`, `Developpee`	**fusion** sur `code_banque` (jointure externe gauche) : actionnaire de référence, part du capital, catégorie, groupe, date de la source (`date_actionnariat`)

Pourquoi l'actionnariat actuel seulement : la dimension a une ligne par banque (relation 1 → plusieurs vers les faits) ; l'historique reste dans `Banques_Actionnariat`.

2.9 `AdresseDepot` (paramètre) et lecture par URL

Texte : `https://raw.githubusercontent.com/AhmedBenArfa/Analyse-secteur-bancaire-tunisien/main`, l'adresse des fichiers bruts (« raw ») du dépôt public. Les fichiers versionnés à la racine (`operations_titres.csv`, `evenements.csv`) sont lus par `Web.Contents(AdresseDepot, [RelativePath = "…"])` : l'adresse de base reste fixe, ce qui permet aussi l'actualisation dans le service Power BI. **Pourquoi une URL** : c'est la troisième source demandée (dossier, Excel, URL) ; le rapport lit la version publiée de ces fichiers, la même pour tout le monde.

2.10 `Operations_Titres` (intermédiaire, non chargée)

63 opérations sur titres (dividendes, attributions gratuites, division du nominal, souscription), une ligne par banque et par date d'effet, avec le cours de référence publié par la Bourse ce jour-là et la clôture de la veille (spécification 7.5).

Étape	Ce qu'elle fait
`Source`, `EnTetes`, `Types`	lit le CSV sur GitHub ; types avec les paramètres régionaux `en-US`
`Facteur`	**colonne calculée** `facteur` = cours de référence publié / clôture de la veille (vide si l'un manque) : l'ajustement fait par la Bourse elle-même, valable pour tous les types d'opération
`Capital`	**colonne conditionnelle** `capital` : vrai pour tout sauf un dividende seul
`FacteurPrix`	facteur de la série « prix » : 1 pour un dividende seul ; pour une attribution le même jour qu'un dividende (BT 10/05/2022), le dividende est retiré de la clôture de la veille avant la parité (4,930 / (6,190 − 0,280) = 0,834, proche de la parité 5/6)
`EnMemoire`	garde la table en mémoire (`Table.Buffer`) : les cumuls la parcourent pour chaque ligne sans relire le fichier en ligne
`CumulPrix`, `CumulRendement`	**facteurs cumulés** par banque : produit des facteurs de cette opération et des suivantes ; série « prix » (opérations sur le capital) et série « rendement total » (dividendes compris)
`Triees`	tri par banque et date

Comment ils serviront : dans `Fait_Cours`, une séance prend le facteur cumulé de la première opération qui la suit ; cours ajusté = clôture × ce facteur. Exemple BIAT : clôture 118,000 le 02/01/2019 et 111,000 le 31/12/2025, soit −5,9 % en brut et +125,8 % ajusté des opérations sur le capital (attributions de 2020 et 2024, division du nominal de 2021).

2.11 `Evenements` (chargée)

11 lignes : une par événement et par banque concernée (don au Fonds 1818, accord salarial APTBEF–FGBEF, moratoire Covid de l'UIB, retraitement BH 2024, déchéance fiscale BNA 2025), avec la date (fin du trimestre concerné), l'indicateur, la description, le montant cité par le document, le fichier et la page. Étapes : lecture du CSV

sur GitHub, types (`en-US`). Elles annoteront les graphiques (marqueurs et info-bulles) une fois les relations créées (étape du modèle).

2.12 `Fait_Cours` (chargée)

Une ligne par banque et par séance (23 460 lignes, du 02/01/2019 au 20/10/2026) : cours non ajustés de la Bourse et cours ajustés des opérations sur titres.

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`, `Types`	lit `cours_banques.csv` ; types avec les paramètres régionaux `en-US` (un cours absent reste vide)
`VidesEnNull`, `SansIsin`	statut vide → `null` ; retire le code ISIN (la banque et le ticker suffisent)
`Operations`	colonnes utiles de `Operations_Titres`, gardées en mémoire (le fichier GitHub est lu une fois)
`Fusion`	**fusion** avec `Operations_Titres` sur la banque : chaque séance reçoit les opérations de sa banque
`Suivante`, `Developpee`	première opération **postérieure** à la séance ; ses facteurs cumulés (produit des facteurs de toutes les opérations qui suivent la séance) deviennent `facteur_prix` et `facteur_rendement`
`FacteurUn`	aucune opération après la séance : facteur 1
`AjustePrix`, `AjusteRendement`	`cloture_ajustee_prix` = clôture × facteur prix (attributions, division du nominal) ; `cloture_ajustee_rendement` = clôture × facteur rendement (dividendes compris)
`SansTransaction`	**colonne conditionnelle** : vrai si le statut contient « sans transaction »
`Triees`, `ParBanque`, `Seances`	**regroupement par banque** : dans chaque banque, triée par date, une clôture vide reprend la dernière clôture connue, puis `rendement` = clôture ajustée (rendement total) / clôture ajustée de la séance précédente − 1 ; la première séance n'a pas de rendement
`TypesFinaux`	types rétablis après le regroupement

Pourquoi ajuster : sans ajustement, une attribution gratuite ou une division du nominal ressemble à une chute du cours. BIAT : 118,000 le 02/01/2019, 111,000 le 31/12/2025, soit −5,9 % en brut et +125,8 % ajusté (49,167 → 111,000). **Pourquoi le rendement sur la série « rendement total »** : le jour du détachement d'un dividende, le cours baisse du montant versé ; ce n'est pas une variation de valeur pour l'actionnaire, la volatilité ne doit pas la compter. Après ajustement, la plus forte variation d'une séance à l'autre est de 10,7 % (BTE, 24/12/2020) : aucune opération ne crée de saut.

2.12b `Variations_Comparables` (chargée)

Une ligne par banque × trimestre × indicateur × mesure : `valeur_N` (la période) et `valeur_N1` (la même période un an plus tôt) **lues dans le même document**, donc dans la même méthode (10 981 lignes).

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`, `Types`	lit `variations_comparables.csv` ; types `en-US`
`VidesEnNull`	mention de retraitement vide → `null`
`Fusion`, `Nature`	**fusion avec `Dim_Indicateur`** : nature de l'indicateur
`Retenir`, `SansTrimDesStocks`	**colonne conditionnelle** et filtre : même règle que `Fait_Indicateurs`, un stock n'a pas de trimestre seul (304 lignes écartées)
`VariationPct`	**colonne calculée** `Variation_Pct` = (N − N-1) / |N-1| (vide si N-1 = 0) ; la valeur absolue garde une hausse positive quand N-1 est négatif (commissions encourues de BTE, autres produits) ; calculée ici, la colonne du CSV n'est pas reprise
`SansColonnes`	retire la variation du CSV, l'année et le rang du trimestre (ils viendront de la table de dates), `nature` et `retenir`

Étape	Ce qu'elle fait
`Periodes`, `Segment`	**segment d'actionnariat** : périodes de `Banques_Actionnariat` gardées en mémoire, puis colonne `Segment_Actionnariat` = fonction `Actionnariat_A_La_Date` (catégorie de la banque à la date d'arrêté, section 2.12h)

Pourquoi une table à part : les niveaux de `Fait_Indicateurs` de deux années viennent de deux documents révisés différents ; leur rapport mélangerait deux méthodes. BH : PNB 2025 −2,8 % dans le document du T4 2025 (2024 retraité), −11,0 % si l'on rapprochait le PNB 2025 du PNB 2024 publié un an plus tôt. Les mesures de croissance (étape DAX) sommeront `valeur_N` et `valeur_N1` puis feront le rapport, pour une banque comme pour le secteur.

2.12c `Fait_Etats_Financiers` (chargée)

Une ligne par banque × date × code (504 lignes) : résultat net avant modifications comptables (`RES_NET`), total de l'actif (`TOTAL_ACTIF`) et capitaux propres du bilan (`CAP_PROPRES_EF`), comptes individuels, au 30/06 de 2020 à 2026 et au 31/12 de 2019 à 2025.

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`, `Types`, `VidesEnNull`	lit `etats_financiers_versions.csv` ; types `en-US` ; textes vides → `null`
`VersionRetenue`	**filtre sur la version retenue** (`version = source_valeur`) : la révisée par le document suivant de même type quand elle existe, sauf exception écrite (BNA 30/06/2025 : résultat avant modifications comptables)
`Colonnes`, `Renommees`	garde la valeur retenue (renommée `valeur_mdt`), la version, le fichier, la page, les statuts et la remarque
`Annualisation`	**colonne conditionnelle** `facteur_annualisation` : 2 pour un semestre, 1 pour un exercice ; le ROE et le ROA multiplieront le résultat par ce facteur
`Periodes`, `Segment`	**segment d'actionnariat** : périodes de `Banques_Actionnariat` gardées en mémoire, puis colonne `Segment_Actionnariat` = fonction `Actionnariat_A_La_Date` (catégorie de la banque à la date d'arrêté, section 2.12h)

Pourquoi la version révisée : le capital et le total de l'actif d'ouverture (31/12/N-1) viennent alors du document de l'exercice N, dans la même méthode que la valeur de fin de période. Les deux dates sans document lisible (BH 31/12/2019, STB 30/06/2023) sont reprises de la colonne N-1 du document suivant (statut « repris du document suivant »).

2.12d `Revisions_Etats_Financiers` (chargée)

Les 423 versions écartées des états financiers : la valeur publiée dans le document de la date (version initiale) quand le document suivant de même type la redonne autrement, et la version révisée non retenue de BNA au 30/06/2025. Mêmes étapes que `Revisions` : lecture du même fichier que `Fait_Etats_Financiers`, **filtre inverse** (`version <> source_valeur`), `Ecart_MDT` = valeur retenue − valeur de la version et `Ecart_Pct` calculés dans Power Query (la colonne du CSV n'est pas reprise), **colonne conditionnelle** `Type_Revision` (statut en clair). Exemple : résultat net de l'UIB au 30/06/2020, −41,182 MDT publié, 17,912 retenu (+59,094 : intérêts du moratoire Covid réintégrés).

2.12e `Controle` (chargée)

Une ligne par document trimestriel (336) : statut de l'extraction, nombre de valeurs lues, alertes, fichier, gabarit de mise en page.

Étape	Ce qu'elle fait
`Fichier`, `Source`, `EnTetes`, `Types`, `VidesEnNull`	lit `controle_extraction.csv` ; textes vides → `null`
`DateArrete`	**colonne calculée** `date_arrete` : fin du trimestre à partir du texte « 2024T3 » (année = 4 premiers caractères, mois = 3 × rang du trimestre, dernier jour du mois) ; servira à la relation avec la table des dates
`NbAlertes`	nombre d'alertes du document (elles sont séparées par « | ») ; 0 sans alerte
`CategorieStatut`	**colonne conditionnelle** : conforme (`ok`) ; écart signalé, qui vient de la banque et ne bloque pas (`retraitement possible`, `ecart trimestre/cumul`, `ecart composantes`) ; tableau en image saisi à la main ; tableau en image repris du document suivant ; alerte bloquante (2 erreurs du document de la banque, vérifiées sur l'image)

Pourquoi : la page qualité montre, pour chaque banque et chaque trimestre, comment le chiffre a été obtenu ; les statuts techniques du script deviennent cinq catégories lisibles.

2.12f `Ecarts_Annuels` (intermédiaire) et `Controle_Annuel` (chargée)

`Controle_Annuel` compare, pour chaque banque, année (2020–2025) et indicateur de flux (PNB, produits et charges d'exploitation, charges opératoires), la somme des 4 trimestres retenus au cumul annuel retenu (288 lignes). `Ecarts_Annuels` donne la cause de chacun des 22 écarts, trouvée dans les PDF.

Requête	Étape	Ce qu'elle fait
`Ecarts_Annuels`	`Source`	lit `docs\ecarts_annuels.csv` dans le dossier du projet (`File.Contents(CheminDonnees & …)`) : fichier versionné, examiné à la main, hors de `donnees_publiees`
`Ecarts_Annuels`	`Types`, `Colonnes`	types `en-US` ; garde la clé (banque, année, indicateur) et l'explication (catégorie, cause, fichier, page, correction)
`Controle_Annuel`	`Fichier` … `Types`	lit `controle_annuel.csv`
`Controle_Annuel`	`Ecart`	**colonne calculée** `Ecart_MDT` = somme des trimestres − cumul annuel (la colonne du CSV n'est pas reprise)
`Controle_Annuel`	`DateArrete`	31/12 de l'année, pour la relation avec les dates
`Controle_Annuel`	`Fusion`, `Developpee`	**fusion sur trois colonnes** (banque, année, indicateur) avec `Ecarts_Annuels` : catégorie, cause, fichier et page de la preuve, correction
`Controle_Annuel`	`StatutClair`	**colonne conditionnelle** : statut en clair ; un « ecart » sans cause écrite serait affiché « écart sans cause écrite » (aucun aujourd'hui, `test_ecarts_annuels_expliques` le garantit)

Pourquoi : un écart entre la somme des trimestres et l'année n'est jamais corrigé en silence ; la page qualité montre sa cause et le document qui la prouve (ex. ATB 2021 : T2 retraité dans la colonne du trimestre mais pas dans le cumul du semestre).

2.12g `Gabarits` (intermédiaire), `Definitions_Charges` et `Definitions_Capitaux_Propres` (chargées)

`gabarits.csv` (versionné à la racine) décrit la mise en page des documents de chaque banque ; deux de ses colonnes disent ce que la banque met dans ses charges opératoires et dans ses capitaux propres trimestriels.

Requête	Étape	Ce qu'elle fait
`Gabarits`	`Source`	lit `gabarits.csv` dans le dossier du projet (`File.Contents`), **11 colonnes** : la remarque, dernière colonne, contient parfois un « ; »

Requête	Étape	Ce qu'elle fait
`Gabarits`	`Colonnes`, `Types`	garde la banque, la période et les deux définitions
`Definitions_Charges`	`Periodes`	**regroupement** par banque et définition : premier et dernier trimestre (`List.Min`, `List.Max`) ; les gabarits découpent l'historique par mise en page, la définition garde sa période entière
`Definitions_Charges`	`Description`	**colonne conditionnelle** : la définition en clair (sans dotations ; avec dotations publiées ; avec un reste non détaillé)
`Definitions_Capitaux_Propres`	`Colonnes`, `Distinctes`	une ligne par banque (`Table.Distinct`)
`Definitions_Capitaux_Propres`	`Description`	**colonne conditionnelle** : hors résultat de la période ; résultat inclus ; hors résultat avec affectation au 31/12

Pourquoi : le coefficient d'exploitation publié (charges opératoires / PNB) ne se compare pas d'une banque à l'autre (BT et BH sans dotations, UIB et Wifak avec, les autres avec un reste non détaillé) ; le rapport classe les banques avec le coefficient harmonisé et affiche la définition de chaque banque en info-bulle. De même, les capitaux propres trimestriels ne servent qu'à la fiche banque, avec leur définition ; les comparaisons utilisent ceux du bilan.

2.12h Segment d'actionnariat : fonction `Actionnariat_A_La_Date`

L'actionnariat d'une banque peut changer (UBCI : BNP Paribas jusqu'au 15/03/2021, groupe Carte ensuite). Un trimestre doit donc être classé selon l'actionnariat **à sa date**, pas selon l'actionnariat d'aujourd'hui (`Dim_Banque`).

`Actionnariat_A_La_Date` est une **fonction personnalisée** (requête non chargée) : elle reçoit la table des périodes, une banque et une date, garde la ligne de `Banques_Actionnariat` dont la période couvre la date (début vide ou antérieur, fin vide ou postérieure) et renvoie sa catégorie ; vide si aucune ou plusieurs lignes la couvrent (`tests/test_actionnariat.py` vérifie que les périodes se suivent sans trou ni chevauchement). `Fait_Indicateurs`, `Variations_Comparables` et `Fait_Etats_Financiers` l'appellent pour chaque ligne (étapes `Periodes` et `Segment`) : la table des périodes est gardée en mémoire une fois par requête, le fichier Excel n'est pas relu à chaque ligne.

Pourquoi une fonction : la même règle sert à trois tables de faits ; écrite une fois, elle ne peut pas diverger d'une table à l'autre. Résultat : au T4 2020, 4 banques sous contrôle étranger (ATB, Attijari, UBCI, UIB), 3 au T3 2026 ; 3 publiques (BNA, BH, STB) ; 1 mixte (BTE).

2.13 Fonctions et mots-clés M utilisés

Chaque fonction du langage M (Power Query) employée dans les requêtes, ce qu'elle fait ici et où elle sert. Le test `tests/test_guide_powerbi.py` vérifie qu'aucune fonction d'un fichier `powerquery/*.pq` ne manque à ce tableau.

Lire les sources

Fonction	Ce qu'elle fait	Requêtes
`Folder.Files`	liste les fichiers d'un dossier (nom, contenu binaire, extension, dossier, dates) : connecteur Dossier	`Fichiers_Donnees`
`File.Contents`	lit un fichier du disque en binaire	`Banques_Actionnariat`
`Excel.Workbook`	ouvre un classeur Excel et liste ses feuilles et ses tables nommées : connecteur Excel	`Banques_Actionnariat`
`Web.Contents`	télécharge une adresse web ; avec `RelativePath`, l'adresse de base reste fixe : source URL	`Operations_Titres`, `Evenements`
`Csv.Document`	lit un contenu CSV (séparateur `;`, encodage 65001 = UTF-8) en table de textes	`Versions`, `Dim_Indicateur`, `Dim_Banque`, `Operations_Titres`, `Evenements`, `Fait_Cours`

Fonction	Ce qu'elle fait	Requêtes
`QuoteStyle.Csv`	option de `Csv.Document` : un texte entre guillemets peut contenir un `;`	mêmes requêtes
`Table.PromoteHeaders`	la première ligne devient les noms de colonnes	mêmes requêtes

Typer et nettoyer

Fonction	Ce qu'elle fait	Requêtes
`Table.TransformColumnTypes`	donne un type à chaque colonne ; avec la culture `"en-US"`, `29.585` est lu 29,585 (et pas 29 585)	toutes les requêtes qui lisent un fichier
`Table.TransformColumns`	applique une fonction à chaque valeur d'une colonne (ici : contrôle de la catégorie d'actionnariat, erreur sinon)	`Banques_Actionnariat`
`Table.ReplaceValue`	remplace une valeur dans des colonnes (texte vide → `null`, `null` → « non », `null` → 1)	`Versions`, `Fait_Indicateurs`, `Fait_Cours`
`Replacer.ReplaceValue`	option de `Table.ReplaceValue` : remplace la valeur entière (pas un morceau de texte)	mêmes requêtes
`Table.SelectColumns`	garde les colonnes nommées, dans cet ordre	`Fichiers_Donnees`, `Fait_Indicateurs`, `Revisions`, `Fait_Cours`
`Table.RemoveColumns`	retire des colonnes	`Dim_Banque`, `Fait_Indicateurs`, `Fait_Cours`
`Table.RenameColumns`	renomme des colonnes	`Fichiers_Donnees`, `Fait_Indicateurs`
`Table.Sort`	trie les lignes	`Operations_Titres`, `Fait_Cours`
`Order.Ascending`	option de `Table.Sort` : ordre croissant	mêmes requêtes

Filtrer et calculer

Fonction	Ce qu'elle fait	Requêtes
`Table.SelectRows`	garde les lignes qui vérifient une condition (version retenue, versions écartées, ligne en cours, opérations postérieures)	`Fichiers_Donnees`, `Fait_Indicateurs`, `Revisions`, `Dim_Banque`, `Operations_Titres`, `Fait_Cours`
`Table.AddColumn`	ajoute une colonne calculée ligne par ligne : colonne conditionnelle (`if … then … else`) ou calcul	`Dim_Indicateur`, `Fait_Indicateurs`, `Revisions`, `Operations_Titres`, `Fait_Cours`
`Table.AddIndexColumn`	numérote les lignes (0, 1, 2…) pour retrouver la séance précédente	`Fait_Cours`
`Table.FillDown`	recopie vers le bas la dernière valeur non vide (dernière clôture connue)	`Fait_Cours`
`Table.First`	première ligne d'une table, ou une valeur par défaut si elle est vide (première opération postérieure)	`Fait_Cours`
`Table.RowCount`	nombre de lignes d'une table (une seule période d'actionnariat doit couvrir la date)	`Actionnariat_A_La_Date`
`Text.Contains`	le texte contient-il ce mot (« sans transaction ») ?	`Fait_Cours`
`Text.StartsWith`	le texte commence-t-il par ce préfixe (« revisee_31-12 », « exclue ») ?	`Revisions`

Fonction	Ce qu'elle fait	Requêtes
`Text.Proper`	met une majuscule au début du mot (« trim » → « Trim »)	`Fait_Indicateurs`
`Number.Abs`	valeur absolue (écart en % rapporté à la valeur absolue)	`Revisions`, `Variations_Comparables`, `Revisions_Etats_Financiers`
`Number.From`	convertit un texte en nombre (« 2024 » → 2024)	`Controle`
`Text.Start`	premiers caractères d'un texte (« 2024T3 » → « 2024 »)	`Controle`
`Text.End`	derniers caractères d'un texte (« 2024T3 » → « 3 »)	`Controle`
`Date.EndOfMonth`	dernier jour du mois d'une date (date d'arrêté du trimestre)	`Controle`
`Text.Split`	découpe un texte selon un séparateur (alertes séparées par « | »)	`Controle`
`List.Count`	nombre d'éléments d'une liste (nombre d'alertes)	`Controle`
`List.Contains`	la liste contient-elle cette valeur (les quatre catégories d'actionnariat) ?	`Banques_Actionnariat`
`List.Product`	produit des nombres d'une liste (facteurs d'ajustement cumulés)	`Operations_Titres`
`List.Min`	plus petite valeur d'une liste (premier trimestre d'une définition)	`Definitions_Charges`
`List.Max`	plus grande valeur d'une liste (dernier trimestre d'une définition)	`Definitions_Charges`
`List.RemoveNulls`	retire les valeurs vides d'une liste (un facteur absent n'annule pas le produit)	`Operations_Titres`

Combiner et restructurer

Fonction	Ce qu'elle fait	Requêtes
`Table.NestedJoin`	**fusion** de deux requêtes sur une clé : chaque ligne reçoit une table des lignes correspondantes	`Fait_Indicateurs`, `Dim_Banque`, `Fait_Cours`
`JoinKind.LeftOuter`	jointure externe gauche : toutes les lignes de la première table sont gardées	mêmes requêtes
`Table.ExpandTableColumn`	développe la table de la fusion en colonnes (nature, actionnariat)	`Fait_Indicateurs`, `Dim_Banque`
`Table.ExpandRecordColumn`	développe un enregistrement en colonnes (facteurs de l'opération suivante)	`Fait_Cours`
`Table.Combine`	**ajoute** des tables les unes sous les autres (codes des états financiers, séances des 12 banques)	`Dim_Indicateur`, `Fait_Cours`
`Table.UnpivotOtherColumns`	**dépivotage** : une colonne par attribut devient une ligne par attribut	`Fait_Indicateurs`
`Table.Pivot`	**pivot** : une ligne par attribut devient une colonne par attribut	`Fait_Indicateurs`
`Table.Group`	**regroupement** : une table par banque, traitée à part (rendement séance par séance) ; une ligne par banque et définition (période des charges opératoires)	`Fait_Cours`, `Definitions_Charges`

Fonction	Ce qu'elle fait	Requêtes
`Table.Distinct`	garde une seule fois chaque ligne identique (une définition des capitaux propres par banque)	`Definitions_Capitaux_Propres`
`Table.Buffer`	garde une table en mémoire : elle n'est pas relue (ni retéléchargée) à chaque utilisation	`Operations_Titres`, `Fait_Cours`
`List.Buffer`	même chose pour une liste (clôtures connues d'une banque)	`Fait_Cours`

Mots-clés du langage

Mot-clé	Ce qu'il fait
`let … in`	suite d'étapes nommées ; la requête renvoie l'étape écrite après `in` (chaque nom est une ligne de « Étapes appliquées »)
`each [colonne]`	fonction appliquée à chaque ligne ; `[colonne]` est la valeur de la ligne en cours
`(op) => …`	fonction avec un paramètre nommé, quand deux lignes sont comparées (une opération et les autres, une séance et les opérations)
`(periodes as table, banque as text, jour as date) as nullable text => …`	**fonction personnalisée** enregistrée comme requête : paramètres typés, résultat typé ; appelée ligne par ligne par d'autres requêtes
`table{0}[colonne]`	valeur d'une colonne dans la première ligne d'une table
`if … then … else`	condition : c'est le code d'une colonne conditionnelle
`error "…"`	provoque une erreur visible à l'actualisation (catégorie d'actionnariat inconnue)
`#table(type table […], {…})`	table écrite dans la requête (3 codes des états financiers)
`#date(année, mois, jour)`	date écrite à partir de nombres (premier jour du dernier mois du trimestre)
`meta [IsParameterQuery = true …]`	fait d'une valeur un paramètre (`CheminDonnees`, `AdresseDepot`)
`type number`, `type text`, `type date`, `Int64.Type`, `type logical`	types des colonnes : nombre décimal, texte, date, entier, vrai ou faux
`null`	valeur absente ; un calcul avec `null` donne `null` (jamais zéro)

3. Modèle et mesures DAX

Le modèle se construit par groupes, chacun vérifié dans Power BI Desktop avant le suivant : table des dates, relations du modèle en étoile, puis mesures (bases, totaux du secteur, ratios, croissances, ROE et ROA, Bourse, qualité). Le code DAX est dans `dax/` (copie lisible, identique au code du projet `.pbip`) ; une table calculée s'écrit dans le TMDL avec `ecrire_table_calculee` de `scripts/ecrire_tmdl_table.py` (`python scripts/ecrire_tmdl_table.py Dim_Date` pour la table des dates).

3.1 `Dim_Date` (table calculée DAX)

Une table de dates sert de filtre commun à toutes les tables qui ont une date (indicateurs, variations, états financiers, cours, révisions, contrôles, événements) : un segment « 2024T3 » ou « Covid » filtre alors toutes les pages de la même façon, et les fonctions de temps de DAX (`LASTNONBLANK`, `DATEADD`…) ont besoin d'une colonne de dates continue, sans jour manquant. Elle est écrite en DAX plutôt qu'en Power Query parce qu'elle ne lit aucune source : elle se calcule dans le modèle (`dax/Dim_Date.dax`). Le code part de celui de la spécification (section 6), avec deux changements : le libellé du trimestre au format des faits et la colonne de tri de la période.

Étapes :

1. `CALENDAR ( DATE ( 2019, 1, 1 ), DATE ( 2026, 12, 31 ) )` : une ligne par jour, colonne `Date`. Bornes fixes : les données vont du 02/01/2019 (premier cours) au 31/12/2026 au plus ; des bornes fixes évitent qu'une date erronée dans un fait étire la table (ce que ferait `CALENDARAUTO`). Les trimestres 2019T1 à 2019T3 restent dans la table pour les cours de 2019.
2. `ADDCOLUMNS` ajoute à chaque jour :
   - `Annee` (`YEAR`) et `Trimestre` (« T1 » à « T4 », `QUARTER`) : la mesure du chiffre annuel prend le cumul au trimestre « T4 » (spécification 7.1) ;
   - `Annee_Trimestre` : « 2024T3 », **au même format que la colonne `trimestre` des faits et des documents** (`BIAT_2024T3.pdf`), plutôt que le « 2024-T3 » de la première version de la spécification : un même libellé partout ;
   - `Cle_Trimestre` : 20243 (= 2024 × 10 + 3), nombre qui suit l'ordre chronologique ; `Annee_Trimestre` est **triée par** cette colonne (un texte se trierait bien ici aussi, mais le tri explicite ne dépend pas du format du libellé) ;
   - `Fin_Trimestre` : vrai le dernier jour d'un trimestre (31/03, 30/06, 30/09, 31/12), seuls jours où il y a des indicateurs trimestriels et des états financiers ;
   - `Periode` (colonne conditionnelle, `SWITCH ( TRUE (), … )` : la première condition vraie l'emporte) : « Avant Covid » avant le 01/03/2020, « Covid » jusqu'au 31/12/2021, « Après Covid » ensuite. Pour les indicateurs, datés de la fin du trimestre : 2019T4 avant Covid, 2020T1 à 2021T4 Covid (8 trimestres), 2022T1 à 2026T3 après Covid (19 trimestres) ;
   - `Ordre_Periode` (1, 2, 3) : `Periode` est triée par cette colonne, sinon l'ordre alphabétique mettrait « Après Covid » en premier dans les segments et les axes.
3. **Réglages du modèle** (dans le TMDL) : table **marquée comme table de dates** sur la colonne `Date` (`dataCategory: Time`, `isKey` ; dans Power BI : **Outils de table › Marquer comme table de dates**) ; colonnes de tri `Cle_Trimestre` et `Ordre_Periode` **masquées** (elles ne servent qu'au tri) ; synthèse « Ne pas résumer » partout (une année ne s'additionne pas).

Contrôles attendus (vue **Données**, table `Dim_Date`) :

Contrôle	Attendu
Nombre de lignes (bas de la fenêtre)	2 922 (8 années, dont 2020 et 2024 bissextiles)
`Fin_Trimestre` = Vrai (filtre de la colonne)	32 lignes, du 31/03/2019 au 31/12/2026
`Periode` (filtre de la colonne, ou visuel Table avec le nombre de `Date`)	Avant Covid 425, Covid 671, Après Covid 1 826
Ordre dans un segment `Periode`	Avant Covid, Covid, Après Covid
`Annee_Trimestre`	32 valeurs, de 2019T1 à 2026T4, dans l'ordre chronologique
Outils de table › Marquer comme table de dates	coché, colonne `Date`
Volet **Données**	`Cle_Trimestre` et `Ordre_Periode` absentes (masquées)

La table ne filtre les faits qu'à travers les relations (3.2).

3.2 Relations du modèle en étoile

Un modèle en étoile relie chaque table de faits à des **dimensions** (dates, banques, indicateurs) : une ligne de la dimension correspond à plusieurs lignes du fait (relation **1 → plusieurs**), et le filtre ne va que de la dimension vers le fait (**filtre simple**). Un segment de banque, de trimestre ou d'indicateur filtre alors toutes les tables de la même façon, sans relation entre les faits eux-mêmes (deux faits ne se filtrent jamais directement : pas de chemin ambigu). Les relations sont écrites dans `definition/relationships.tmdl` (`python scripts/ecrire_tmdl_table.py relations`) ; 28 relations :

Dimension (côté 1)	Tables de faits (côté plusieurs)	Nombre
`Dim_Date[Date]`	`date_arrete` de `Fait_Indicateurs`, `Variations_Comparables`, `Fait_Etats_Financiers`, `Revisions`, `Revisions_Etats_Financiers`, `Controle`, `Controle_Annuel` ; `Evenements[date]` ; `Fait_Cours[date]`	9
`Dim_Banque[code_banque]`	`banque` des 9 mêmes tables, de `Definitions_Charges` et de `Definitions_Capitaux_Propres` ; `Banques_Actionnariat[code_banque]`	12
`Dim_Indicateur[code_indicateur]`	`code_indicateur` de `Fait_Indicateurs`, `Variations_Comparables`, `Revisions`, `Controle_Annuel` ; `Evenements[indicateur]` ; `code` de `Fait_Etats_Financiers` et `Revisions_Etats_Financiers`	7

Choix :

- **`Fait_Cours` relié par la banque** (`code_banque` → `banque`) et non par le ticker comme dans la première version de la spécification : `Fait_Cours` contient aussi le code de la banque, et une même clé partout est plus simple à lire (une dimension n'a de toute façon qu'une relation active vers une table).
- **`Banques_Actionnariat` et les définitions** (charges opératoires, capitaux propres) reliées à `Dim_Banque` : le segment de banque de la fiche banque les filtre aussi.
- **Codes des états financiers** (`RES_NET`, `TOTAL_ACTIF`, `CAP_PROPRES_EF`) : ajoutés à `Dim_Indicateur` dans Power Query (2.4), ce qui permet la relation avec `Fait_Etats_Financiers` et garde les événements sur le résultat net (moratoire UIB 2020).
- **Contrôle d'intégrité avant d'écrire** : chaque banque, code et date des faits existe dans sa dimension (aucune clé vide ; dates du 02/01/2019 au 20/10/2026). Sinon Power BI ajoute une ligne « (vide) » à la dimension, qui regroupe les lignes orphelines.

Contrôles attendus :

Contrôle	Attendu
Vue **Modèle**, ou **Accueil › Gérer les relations**	28 relations, toutes actives, cardinalité « plusieurs à un (*:1) », filtre « Unique »
Visuel Table : `Dim_Date[Periode]` + nombre de `Fait_Cours[date]`	Avant Covid 3 480, Covid 5 484, Après Covid 14 496 (total 23 460)
Même visuel, segment `Dim_Banque[code_banque]` = BIAT	Covid 457 séances
Visuel Table : `Dim_Date[Periode]` + nombre de `Evenements[id_evenement]`	Covid 9, Après Covid 2 (pas de ligne Avant Covid)
Visuel Table : `Dim_Date[Periode]` + nombre de `Controle[banque]`	Avant Covid 12, Covid 96, Après Covid 228
Visuel Table : `Dim_Indicateur[code_indicateur]` + nombre de `Fait_Etats_Financiers[valeur_mdt]`	`RES_NET`, `TOTAL_ACTIF`, `CAP_PROPRES_EF` : 168 chacun
Segment de banque ou de période dans ces visuels	aucune ligne « (vide) »

3.9 Fonctions DAX utilisées

Chaque fonction DAX employée dans `dax/*.dax`, ce qu'elle fait ici et où elle sert. Le test `tests/test_guide_powerbi.py` vérifie qu'aucune fonction d'un fichier `dax/*.dax` ne manque à ce tableau.

Fonction	Ce qu'elle fait	Code
`CALENDAR`	table d'une colonne `Date`, un jour par ligne entre deux dates	`Dim_Date`
`DATE`	date écrite à partir de l'année, du mois et du jour	`Dim_Date`
`ADDCOLUMNS`	ajoute des colonnes calculées à chaque ligne d'une table	`Dim_Date`

Fonction	Ce qu'elle fait	Code
`YEAR`, `MONTH`, `QUARTER`	année, mois (1 à 12), trimestre (1 à 4) d'une date	`Dim_Date`
`EOMONTH`	dernier jour du mois d'une date (décalé de n mois)	`Dim_Date`
`MOD`	reste de la division entière (mois multiple de 3 = fin de trimestre)	`Dim_Date`
`SWITCH`	renvoie la valeur de la première condition vraie ; avec `TRUE ()`, sert de colonne conditionnelle	`Dim_Date`
`TRUE`	valeur vraie	`Dim_Date`

4. Pages du rapport : visuels, champs et mesures

À préparer quand les mesures seront écrites (liste des pages : spécification, section 9).

Annexe A — Code M de chaque requête

Texte exact des fichiers `powerquery/*.pq`, identique au code du projet `.pbip`, dans l'ordre de Power Query.

CheminDonnees

```
// Auteur : Ahmed Ben Arfa
// Paramètre : dossier du projet sur ce poste (contient donnees\, donnees_reference\ et docs\).
// À changer une seule fois si le projet est copié ailleurs ; toutes les requêtes en dépendent.
"D:\Projet\Analyse-secteur-bancaire-tunisien" meta [IsParameterQuery = true, Type = "Text",
IsParameterQueryRequired = true]
```

AdresseDepot

```
// Auteur : Ahmed Ben Arfa
// Paramètre : adresse des fichiers bruts (« raw ») du dépôt GitHub public, branche main. Les fichiers
// versionnés à la racine (operations_titres.csv, evenements.csv) sont lus par cette adresse (source URL).
"https://raw.githubusercontent.com/AhmedBenArfa/Analyse-secteur-bancaire-tunisien/main" meta
[IsParameterQuery = true, Type = "Text", IsParameterQueryRequired = true]
```

Fichiers_Donnees

```
// Auteur : Ahmed Ben Arfa
// Requête intermédiaire (non chargée) : les CSV du dossier versionné donnees_publiees\, lus avec le
// connecteur Dossier. Les autres requêtes y prennent leur fichier par son nom : un seul endroit dépend
// du disque. Le dossier est régénéré par python scripts/publier_donnees.py (donnees\ reste local).
let
    Source = Folder.Files(CheminDonnees & "\donnees_publiees"),
    // seulement les CSV posés directement dans donnees_publiees\ (pas LISEZMOI.md)
    CsvDuDossier = Table.SelectRows(
        Source,
        each [Extension] = ".csv" and [Folder Path] = CheminDonnees & "\donnees_publiees\"
    ),
    Colonnes = Table.SelectColumns(CsvDuDossier, {"Name", "Content", "Date modified"}),
    Renommees = Table.RenameColumns(Colonnes, {{"Name", "fichier"}, {"Content", "contenu"}, {"Date modified",
"modifie_le"}})
in
    Renommees
```

Versions

```
// Auteur : Ahmed Ben Arfa
// Requête intermédiaire (non chargée) : toutes les versions de chaque valeur des indicateurs trimestriels
// (initiale, révisée par le document suivant, 31/12 provisoire), une ligne par banque × trimestre ×
// indicateur × mesure (trim / cumul) × version. Fait_Indicateurs et Revisions en partent.
let
    Fichier = Fichiers_Donnees{[fichier = "indicateurs_versions.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    // le CSV écrit les décimales avec un point : types appliqués avec les paramètres régionaux en-US
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"trimestre", type text}, {"date_arrete", type date},
            {"annee", Int64.Type}, {"num_trimestre", Int64.Type}, {"code_indicateur", type text},
            {"mesure", type text}, {"version", type text}, {"valeur_mdt", type number},
            {"fichier", type text}, {"page", Int64.Type}, {"valeur_retenue", type number},
            {"source_valeur", type text}, {"ecart_vs_retenue", type number},
            {"statut_revision", type text}, {"note_definition", type text},
            {"mention_retraitement", type text}, {"rupture_serie", type text},
            {"detail_rupture", type text}
        },
        "en-US"
    ),
    // cases vides du CSV (valeur non disponible, pas de note) : null, jamais zéro
    VidesEnNull = Table.ReplaceValue(
        Types, "", null, Replacer.ReplaceValue,
        {"statut_revision", "note_definition", "mention_retraitement", "rupture_serie", "detail_rupture"}
    )
in
    VidesEnNull
```

Dim_Indicateur

```
// Auteur : Ahmed Ben Arfa
// Dimension des indicateurs (chargée) : une ligne par code, avec le libellé, la catégorie, la nature
// (flux : s'additionne ; stock : solde à une date, jamais additionné) et l'ordre d'affichage.
// Les 3 codes des états financiers y sont ajoutés pour que les événements qui les concernent gardent
// leur relation (spécification 5.1 et 6). Fait_Indicateurs y prend la nature par une fusion.
let
    Fichier = Fichiers_Donnees{[fichier = "dim_indicateur.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {{"code_indicateur", type text}, {"libelle", type text}, {"categorie", type text},
         {"nature", type text}, {"ordre", Int64.Type}},
        "en-US"
    ),
    // colonne ajoutée : d'où viennent les valeurs de l'indicateur
    SourceIndicateurs = Table.AddColumn(Types, "source", each "indicateurs trimestriels", type text),
    // codes des états financiers (libellés de scripts/etats_financiers.py), à la suite des 24 codes
    EtatsFinanciers = #table(
        type table [code_indicateur = text, libelle = text, categorie = text, nature = text,
                    ordre = Int64.Type, source = text],
        {
            {"RES_NET", "Résultat net (avant modifications comptables)", "Résultat", "flux", 25, "états
financiers"},
            {"TOTAL_ACTIF", "Total de l'actif", "Bilan", "stock", 26, "états financiers"},
            {"CAP_PROPRES_EF", "Total des capitaux propres (bilan, résultat inclus)", "Bilan", "stock", 27,
"états financiers"}
        }
    ),
    Ajoutees = Table.Combine({SourceIndicateurs, EtatsFinanciers})
in
    Ajoutees
```

Fait_Indicateurs

```
// Auteur : Ahmed Ben Arfa
// Table de faits des indicateurs trimestriels (chargée) : la valeur retenue de chaque indicateur, une
// ligne par banque × trimestre × indicateur, avec le trimestre seul et le cumul côte à côte, et pour
// chacun la version retenue, le fichier, la page et la rupture de série (spécification 5.1 et 5.2).
let
    Source = Versions,
    // filtre explicite : seule la version retenue (révisée quand elle existe) entre dans les faits ;
    // les versions écartées vont dans la requête Revisions (page qualité)
    VersionRetenue = Table.SelectRows(Source, each [version] = [source_valeur]),
    Colonnes = Table.SelectColumns(
        VersionRetenue,
        {"banque", "trimestre", "date_arrete", "code_indicateur", "mesure",
         "valeur_retenue", "source_valeur", "fichier", "page", "rupture_serie", "detail_rupture"}
    ),
    // fusion avec Dim_Indicateur : nature de chaque indicateur (flux ou stock)
    Fusion = Table.NestedJoin(
        Colonnes, {"code_indicateur"}, Dim_Indicateur, {"code_indicateur"}, "Dim_Indicateur",
JoinKind.LeftOuter
    ),
    Nature = Table.ExpandTableColumn(Fusion, "Dim_Indicateur", {"nature"}),
    // colonne conditionnelle : un stock est un solde à une date, sans trimestre seul ; la ligne « trim »
    // de certains documents (T1 : copie du solde ; STB 2019T4 et UBCI 2024-2026 : mouvement imprimé dans
    // la colonne du trimestre) est écartée : Valeur_Trim et ses colonnes de source restent vides
    Retenir = Table.AddColumn(
        Nature, "retenir", each if [nature] = "stock" and [mesure] = "trim" then false else true, type
logical
    ),
    SansTrimDesStocks = Table.SelectRows(Retenir, each [retenir]),
    // pas de rupture de série : « non » au lieu d'une case vide (segment lisible) ; le détail reste vide
    RuptureNon = Table.ReplaceValue(SansTrimDesStocks, null, "non", Replacer.ReplaceValue,
{"rupture_serie"}),
    SansColonnesAjoutees = Table.RemoveColumns(RuptureNon, {"nature", "retenir"}),
    // noms des attributs tels qu'ils seront préfixés après le pivot (Valeur_Trim, Source_Cumul…)
    Renommees = Table.RenameColumns(
        SansColonnesAjoutees,
        {{"valeur_retenue", "Valeur"}, {"source_valeur", "Source"}, {"fichier", "Fichier"},
         {"page", "Page"}, {"rupture_serie", "Rupture"}, {"detail_rupture", "Detail_Rupture"}}
    ),
    // dépivotage : une ligne par attribut ; une case vide (pas de détail de rupture) ne donne pas de ligne
    Depivotees = Table.UnpivotOtherColumns(
        Renommees, {"banque", "trimestre", "date_arrete", "code_indicateur", "mesure"}, "attribut", "contenu"
    ),
    // nom de la future colonne : attribut + mesure (« trim » → Trim, « cumul » → Cumul)
    NomColonne = Table.AddColumn(Depivotees, "colonne", each [attribut] & "_" & Text.Proper([mesure]), type
text),
    SansMesure = Table.RemoveColumns(NomColonne, {"mesure", "attribut"}),
    // pivot sans agrégation (une seule valeur par cellule) ; liste fixe : les colonnes existent toujours
    Pivotees = Table.Pivot(
        SansMesure,
        {"Valeur_Trim", "Valeur_Cumul", "Source_Trim", "Source_Cumul", "Fichier_Trim", "Fichier_Cumul",
         "Page_Trim", "Page_Cumul", "Rupture_Trim", "Rupture_Cumul", "Detail_Rupture_Trim",
"Detail_Rupture_Cumul"},
        "colonne",
        "contenu"
    ),
    Types = Table.TransformColumnTypes(
        Pivotees,
        {
            {"Valeur_Trim", type number}, {"Valeur_Cumul", type number},
            {"Source_Trim", type text}, {"Source_Cumul", type text},
            {"Fichier_Trim", type text}, {"Fichier_Cumul", type text},
            {"Page_Trim", Int64.Type}, {"Page_Cumul", Int64.Type},
            {"Rupture_Trim", type text}, {"Rupture_Cumul", type text},
            {"Detail_Rupture_Trim", type text}, {"Detail_Rupture_Cumul", type text}
        }
    ),
    // segment d'actionnariat : catégorie de la banque à la date d'arrêté, selon les périodes de
    // Banques_Actionnariat (fonction Actionnariat_A_La_Date ; UBCI : groupe étranger jusqu'au 15/03/2021)
    Periodes = Table.Buffer(
```

```
        Table.SelectColumns(Banques_Actionnariat, {"code_banque", "date_debut", "date_fin", "actionnariat"})
    ),
    Segment = Table.AddColumn(
        Types, "Segment_Actionnariat", each Actionnariat_A_La_Date(Periodes, [banque], [date_arrete]), type
text
    )
in
    Segment
```

Revisions

```
// Auteur : Ahmed Ben Arfa
// Versions non retenues des indicateurs trimestriels (chargée) : chaque valeur publiée puis remplacée
// (initiale révisée par le document suivant, 31/12 provisoire remplacé par le T4 de l'année suivante,
// colonne fausse écartée, document en image), avec l'écart à la valeur retenue. Page « qualité des
// données » (spécification 5.1 et 5.2). Complément exact de Fait_Indicateurs : même source, filtre inverse.
let
    Source = Versions,
    // filtre explicite, inverse de celui de Fait_Indicateurs : toutes les versions écartées
    VersionsNonRetenues = Table.SelectRows(Source, each [version] <> [source_valeur]),
    Colonnes = Table.SelectColumns(
        VersionsNonRetenues,
        {"banque", "trimestre", "date_arrete", "code_indicateur", "mesure", "version", "valeur_mdt",
         "fichier", "page", "valeur_retenue", "source_valeur", "statut_revision", "note_definition",
         "mention_retraitement"}
    ),
    // écart calculé ici (la colonne ecart_vs_retenue du CSV n'est pas reprise) : de combien la valeur
    // retenue s'écarte de cette version ; vide si l'une des deux manque (document en image, capitaux
    // propres au 31/12 provisoire sans autre version), jamais zéro
    EcartMDT = Table.AddColumn(Colonnes, "Ecart_MDT", each [valeur_retenue] - [valeur_mdt], type nullable
number),
    EcartPct = Table.AddColumn(
        EcartMDT, "Ecart_Pct",
        each if [valeur_mdt] = null or [valeur_mdt] = 0 then null else [Ecart_MDT] /
Number.Abs([valeur_mdt]),
        type nullable number
    ),
    // colonne conditionnelle : d'où vient la version écartée
    OrigineVersion = Table.AddColumn(
        EcartPct, "Origine_Version",
        each if [version] = "initiale" then "document du trimestre"
            else if Text.StartsWith([version], "revisee_31-12") then "31/12 provisoire (document T1 à T3
suivant)"
            else if Text.StartsWith([version], "revisee_T") then "colonne N-1 du document suivant"
            else [version],
        type text
    ),
    // colonne conditionnelle : statut de la révision en clair, pour les segments de la page qualité
    TypeRevision = Table.AddColumn(
        OrigineVersion, "Type_Revision",
        each if [statut_revision] = null then "identique à la valeur retenue (arrondi)"
            else if [statut_revision] = "retraitement_mineur" then "retraitement mineur (2 % au plus)"
            else if [statut_revision] = "revision_importante" then "révision importante (plus de 2 %)"
            else if [statut_revision] = "reclassement" then "reclassement (total inchangé)"
            else if [statut_revision] = "illisible (image)" then "document publié en image"
            else if Text.StartsWith([statut_revision], "exclue (capitaux propres") then "31/12 provisoire
écarté (capitaux propres)"
            else if Text.StartsWith([statut_revision], "exclue") then "colonne fausse du document, écartée"
            else [statut_revision],
        type text
    )
in
    TypeRevision
```

Banques_Actionnariat

```
// Auteur : Ahmed Ben Arfa
// Actionnariat des banques (chargée) : table Excel « Banques » de donnees_reference\banques.xlsx
// (connecteur Excel), une ligne par banque et par période d'actionnaire de référence (date_debut,
// date_fin vides quand l'actionnaire n'a pas changé de 2019 à 2026), avec la source et la page de
// chaque valeur. Règle de classement par le contrôle : spécification 5.3. Dim_Banque en prend la ligne
// en cours ; la fiche banque affiche l'historique et ses sources.
let
    Classeur = Excel.Workbook(File.Contents(CheminDonnees & "\donnees_reference\banques.xlsx"), true),
    // la table Excel nommée (pas la feuille) : ses limites suivent les lignes ajoutées dans Excel
    Banques = Classeur{[Item = "Banques", Kind = "Table"]}[Data],
    Types = Table.TransformColumnTypes(
        Banques,
        {
            {"code_banque", type text}, {"nom", type text}, {"actionnaire_reference", type text},
            {"part_capital_pct", type number}, {"actionnariat", type text}, {"groupe", type text},
            {"date_reference", type date}, {"source", type text}, {"page", type text},
            {"date_debut", type date}, {"date_fin", type date}, {"remarque", type text}
        }
    ),
    // une catégorie hors de la règle (faute de frappe dans Excel) se verrait ici comme une erreur
    CategorieValide = Table.TransformColumns(
        Types,
        {{"actionnariat", each if List.Contains(
            {"public", "groupe étranger", "mixte", "privé sans contrôle étranger"}, _) then _
            else error "catégorie d'actionnariat inconnue : " & (if _ = null then "(vide)" else _), type
text}}
    )
in
    CategorieValide
```

Dim_Banque

```
// Auteur : Ahmed Ben Arfa
// Dimension des banques (chargée) : code, nom, ticker de la Bourse (dim_banque.csv), et actionnariat
// actuel (Banques_Actionnariat, depuis le fichier Excel) : actionnaire de référence, part du capital,
// catégorie (public, groupe étranger, mixte, privé sans contrôle étranger), groupe, date de la source.
let
    Fichier = Fichiers_Donnees{[fichier = "dim_banque.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes, {{"code_banque", type text}, {"nom", type text}, {"ticker_bvmt", type text}}, "en-US"
    ),
    // les colonnes actionnariat et groupe du CSV sont vides : elles viennent du fichier Excel
    SansColonnesVides = Table.RemoveColumns(Types, {"actionnariat", "groupe"}),
    // ligne en cours de chaque banque : celle dont la période n'a pas de fin
    ActionnariatActuel = Table.SelectRows(Banques_Actionnariat, each [date_fin] = null),
    // fusion avec l'actionnariat actuel (une ligne par banque : sinon la fusion dupliquerait la banque)
    Fusion = Table.NestedJoin(
        SansColonnesVides, {"code_banque"}, ActionnariatActuel, {"code_banque"}, "Actionnariat",
JoinKind.LeftOuter
    ),
    Developpee = Table.ExpandTableColumn(
        Fusion, "Actionnariat",
        {"actionnaire_reference", "part_capital_pct", "actionnariat", "groupe", "date_reference"},
        {"actionnaire_reference", "part_capital_pct", "actionnariat", "groupe", "date_actionnariat"}
    )
in
    Developpee
```

Actionnariat_A_La_Date

```
// Auteur : Ahmed Ben Arfa
// Fonction (requête non chargée) : catégorie d'actionnariat d'une banque à une date, lue dans les périodes
// de Banques_Actionnariat (la ligne dont la période couvre la date ; spécification 5.3). Appelée par
// Fait_Indicateurs, Variations_Comparables et Fait_Etats_Financiers (colonne Segment_Actionnariat).
// Vide si aucune période ne couvre la date (tests/test_actionnariat.py vérifie qu'il n'y a pas de trou).
(periodes as table, banque as text, jour as date) as nullable text =>
let
    Couvrantes = Table.SelectRows(
        periodes,
        each [code_banque] = banque
            and ([date_debut] = null or [date_debut] <= jour)
            and ([date_fin] = null or [date_fin] >= jour)
    ),
    Segment = if Table.RowCount(Couvrantes) = 1 then Couvrantes{0}[actionnariat] else null
in
    Segment
```

Operations_Titres

```
// Auteur : Ahmed Ben Arfa
// Requête intermédiaire (non chargée) : opérations sur titres des 12 banques (dividendes, attributions
// gratuites, division du nominal, souscription), lues sur GitHub (source URL), avec le facteur
// d'ajustement publié par la Bourse et les facteurs cumulés qui ajusteront les cours de Fait_Cours
// (spécification 7.5). Cours ajusté d'une séance = cours × produit des facteurs des opérations postérieures.
let
    // Web.Contents avec RelativePath : l'adresse de base reste fixe (actualisation possible en ligne)
    Source = Csv.Document(
        Web.Contents(AdresseDepot, [RelativePath = "operations_titres.csv"]),
        [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]
    ),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    // décimales avec un point : types avec les paramètres régionaux en-US
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"isin", type text}, {"date_effet", type date}, {"types", type text},
            {"details", type text}, {"dividende", type number}, {"date_veille", type date},
            {"cloture_veille", type number}, {"cours_reference_publie", type number},
            {"cours_reference_theorique", type number}, {"ecart_theorique", type number},
            {"bulletin", type text}, {"page_cours", Int64.Type}, {"page_avis", Int64.Type},
            {"extrait_avis", type text}, {"detection", type text}
        },
        "en-US"
    ),
    // facteur de la Bourse : cours de référence publié le jour de l'opération / clôture de la veille ;
    // vide si le cours de référence manque (opération signalée, jamais de facteur inventé)
    Facteur = Table.AddColumn(
        Types, "facteur",
        each if [cours_reference_publie] = null or [cloture_veille] = null then null
            else [cours_reference_publie] / [cloture_veille],
        type nullable number
    ),
    // colonne conditionnelle : opération sur le capital (tout sauf un dividende seul)
    Capital = Table.AddColumn(Facteur, "capital", each [types] <> "dividende", type logical),
    // facteur de la série « prix » : 1 pour un dividende seul ; si un dividende est détaché le même jour
    // qu'une attribution (BT 10/05/2022), il est retiré de la clôture de la veille avant la parité
    FacteurPrix = Table.AddColumn(
        Capital, "facteur_prix",
        each if not [capital] then 1
            else if [facteur] = null then null
            else if [dividende] <> null then [cours_reference_publie] / ([cloture_veille] - [dividende])
            else [facteur],
        type nullable number
    ),
    // table gardée en mémoire : les cumuls la parcourent pour chaque ligne sans relire le fichier en ligne
    EnMemoire = Table.Buffer(FacteurPrix),
    // facteurs cumulés : produit des facteurs de la banque pour cette opération et les suivantes ; une
    // séance antérieure à l'opération prend le cumul de la première opération qui la suit
    CumulPrix = Table.AddColumn(
        EnMemoire, "facteur_cumule_prix",
        (op) => List.Product(List.RemoveNulls(Table.SelectRows(
            EnMemoire, (autre) => autre[banque] = op[banque] and autre[date_effet] >= op[date_effet])
[facteur_prix])),
        type number
    ),
    CumulRendement = Table.AddColumn(
        CumulPrix, "facteur_cumule_rendement",
        (op) => List.Product(List.RemoveNulls(Table.SelectRows(
            EnMemoire, (autre) => autre[banque] = op[banque] and autre[date_effet] >= op[date_effet])
[facteur])),
        type number
    ),
    Triees = Table.Sort(CumulRendement, {{"banque", Order.Ascending}, {"date_effet", Order.Ascending}})
in
    Triees
```

Evenements

```
// Auteur : Ahmed Ben Arfa
// Événements qui expliquent une rupture de série (chargée) : don au Fonds 1818, accord salarial
// APTBEF–FGBEF, moratoire Covid de l'UIB, retraitement BH 2024, déchéance fiscale BNA 2025 ; une ligne par
// événement et par banque, avec le montant cité, le fichier et la page. Lus sur GitHub (source URL) ;
// annotations des graphiques (spécification 5.1, 7.3, pages 3 et 9).
let
    Source = Csv.Document(
        Web.Contents(AdresseDepot, [RelativePath = "evenements.csv"]),
        [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]
    ),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"id_evenement", type text}, {"date", type date}, {"banque", type text}, {"indicateur", type
text},
            {"description", type text}, {"montant_mdt", type number}, {"fichier", type text}, {"page",
Int64.Type}
        },
        "en-US"
    )
in
    Types
```

Fait_Cours

```
// Auteur : Ahmed Ben Arfa
// Table de faits des cours (chargée) : une ligne par banque et par séance du 02/01/2019 à la dernière
// séance collectée, cours non ajustés de la Bourse, cours ajustés des opérations sur titres
// (Operations_Titres), rendement d'une séance à l'autre et marqueur de séance sans transaction
// (spécification 7.5).
let
    Fichier = Fichiers_Donnees{[fichier = "cours_banques.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"ticker", type text}, {"banque", type text}, {"isin", type text}, {"date", type date},
            {"ouverture", type number}, {"haut", type number}, {"bas", type number}, {"cloture", type
number},
            {"volume", Int64.Type}, {"nb_transactions", Int64.Type}, {"capitaux", type number},
            {"statut", type text}, {"source", type text}, {"fichier", type text}
        },
        "en-US"
    ),
    // une case vide du CSV (cours absent, pas de statut) reste vide, jamais zéro
    VidesEnNull = Table.ReplaceValue(Types, "", null, Replacer.ReplaceValue, {"statut"}),
    SansIsin = Table.RemoveColumns(VidesEnNull, {"isin"}),
    // opérations de la banque, gardées en mémoire (lues une fois sur GitHub)
    Operations = Table.Buffer(
        Table.SelectColumns(Operations_Titres, {"banque", "date_effet", "facteur_cumule_prix",
"facteur_cumule_rendement"})
    ),
    // fusion avec Operations_Titres sur la banque : chaque séance reçoit les opérations de sa banque
    Fusion = Table.NestedJoin(SansIsin, {"banque"}, Operations, {"banque"}, "operations",
JoinKind.LeftOuter),
    // première opération postérieure à la séance (les opérations sont triées par date) : ses facteurs
    // cumulés sont le produit des facteurs de toutes les opérations qui suivent la séance
    Suivante = Table.AddColumn(
        Fusion, "operation_suivante",
        (s) => Table.First(Table.SelectRows(s[operations], (o) => o[date_effet] > s[date]), null)
    ),
    Developpee = Table.ExpandRecordColumn(
        Table.RemoveColumns(Suivante, {"operations"}), "operation_suivante",
        {"facteur_cumule_prix", "facteur_cumule_rendement"}, {"facteur_prix", "facteur_rendement"}
    ),
    // aucune opération après la séance : facteur 1 (cours déjà au niveau d'aujourd'hui)
    FacteurUn = Table.ReplaceValue(Developpee, null, 1, Replacer.ReplaceValue, {"facteur_prix",
"facteur_rendement"}),
    // cours ajustés : série « prix » (opérations sur le capital) et série « rendement total » (dividendes
compris)
    AjustePrix = Table.AddColumn(
        FacteurUn, "cloture_ajustee_prix", each [cloture] * [facteur_prix], type nullable number
    ),
    AjusteRendement = Table.AddColumn(
        AjustePrix, "cloture_ajustee_rendement", each [cloture] * [facteur_rendement], type nullable number
    ),
    // colonne conditionnelle : séance sans transaction (mesure de liquidité, avertissement « titre peu
liquide »)
    SansTransaction = Table.AddColumn(
        AjusteRendement, "sans_transaction",
        each if [statut] <> null and Text.Contains([statut], "sans transaction") then true else false, type
logical
    ),
    Triees = Table.Sort(SansTransaction, {{"banque", Order.Ascending}, {"date", Order.Ascending}}),
    // rendement d'une séance à l'autre sur la série « rendement total », banque par banque ; une clôture
    // vide reprend la dernière clôture connue (rendement nul ce jour-là), la première séance n'a pas de
rendement
    ParBanque = Table.Group(
        Triees, {"banque"},
        {{"seances", (t) =>
            let
                Connue = Table.FillDown(
                    Table.AddColumn(t, "cloture_connue", each [cloture_ajustee_rendement], type nullable
```

```
number),
                    {"cloture_connue"}
                ),
                Valeurs = List.Buffer(Connue[cloture_connue]),
                AvecIndex = Table.AddIndexColumn(Connue, "rang", 0, 1, Int64.Type),
                AvecRendement = Table.AddColumn(
                    AvecIndex, "rendement",
                    each if [rang] = 0 or [cloture_connue] = null or Valeurs{[rang] - 1} = null then null
                        else [cloture_connue] / Valeurs{[rang] - 1} - 1,
                    type nullable number
                )
            in
                Table.RemoveColumns(AvecRendement, {"cloture_connue", "rang"}),
          type table}}
    ),
    Seances = Table.Combine(ParBanque[seances]),
    // types rétablis après le regroupement
    TypesFinaux = Table.TransformColumnTypes(
        Seances,
        {
            {"ticker", type text}, {"banque", type text}, {"date", type date},
            {"ouverture", type number}, {"haut", type number}, {"bas", type number}, {"cloture", type
number},
            {"volume", Int64.Type}, {"nb_transactions", Int64.Type}, {"capitaux", type number},
            {"statut", type text}, {"source", type text}, {"fichier", type text},
            {"facteur_prix", type number}, {"facteur_rendement", type number},
            {"cloture_ajustee_prix", type number}, {"cloture_ajustee_rendement", type number},
            {"sans_transaction", type logical}, {"rendement", type number}
        }
    )
in
    TypesFinaux
```

Variations_Comparables

```
// Auteur : Ahmed Ben Arfa
// Variations sur un an (chargée) : pour chaque document, la valeur de la période (valeur_N) et celle
// de la même période un an plus tôt publiée dans le MÊME document (valeur_N1), donc dans la même
// méthode ; une ligne par banque × trimestre × indicateur × mesure (trim / cumul). Source de toutes les
// croissances du rapport : jamais deux documents rapprochés (spécification 5.2 et 7.3).
let
    Fichier = Fichiers_Donnees{[fichier = "variations_comparables.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"trimestre", type text}, {"date_arrete", type date}, {"annee",
Int64.Type},
            {"num_trimestre", Int64.Type}, {"code_indicateur", type text}, {"mesure", type text},
            {"valeur_N", type number}, {"valeur_N1", type number}, {"variation_pct", type number},
            {"fichier", type text}, {"page", Int64.Type}, {"mention_retraitement", type text}
        },
        "en-US"
    ),
    VidesEnNull = Table.ReplaceValue(Types, "", null, Replacer.ReplaceValue, {"mention_retraitement"}),
    // fusion avec Dim_Indicateur : nature de chaque indicateur (flux ou stock)
    Fusion = Table.NestedJoin(
        VidesEnNull, {"code_indicateur"}, Dim_Indicateur, {"code_indicateur"}, "Dim_Indicateur",
JoinKind.LeftOuter
    ),
    Nature = Table.ExpandTableColumn(Fusion, "Dim_Indicateur", {"nature"}),
    // même règle que Fait_Indicateurs : un stock n'a pas de trimestre seul (colonne conditionnelle + filtre)
    Retenir = Table.AddColumn(
        Nature, "retenir", each if [nature] = "stock" and [mesure] = "trim" then false else true, type
logical
    ),
    SansTrimDesStocks = Table.SelectRows(Retenir, each [retenir]),
    // variation recalculée ici (la colonne du CSV n'est pas reprise) : (N − N-1) / |N-1|, pour qu'une hausse
    // reste positive quand la valeur N-1 est négative ; vide si N-1 est nul
    VariationPct = Table.AddColumn(
        SansTrimDesStocks, "Variation_Pct",
        each if [valeur_N1] = null or [valeur_N1] = 0 then null
            else ([valeur_N] - [valeur_N1]) / Number.Abs([valeur_N1]),
        type nullable number
    ),
    // colonnes du CSV inutiles dans le modèle (année et rang du trimestre viendront de la table de dates)
    SansColonnes = Table.RemoveColumns(VariationPct, {"variation_pct", "annee", "num_trimestre", "nature",
"retenir"}),
    // segment d'actionnariat : catégorie de la banque à la date d'arrêté, selon les périodes de
    // Banques_Actionnariat (fonction Actionnariat_A_La_Date ; UBCI : groupe étranger jusqu'au 15/03/2021)
    Periodes = Table.Buffer(
        Table.SelectColumns(Banques_Actionnariat, {"code_banque", "date_debut", "date_fin", "actionnariat"})
    ),
    Segment = Table.AddColumn(
        SansColonnes, "Segment_Actionnariat", each Actionnariat_A_La_Date(Periodes, [banque], [date_arrete]),
type text
    )
in
    Segment
```

Fait_Etats_Financiers

```
// Auteur : Ahmed Ben Arfa
// Table de faits des états financiers (chargée) : résultat net (avant modifications comptables), total de
// l'actif et capitaux propres du bilan des 12 banques, comptes individuels, au 30/06 (2020–2026) et au
// 31/12 (2019–2025) ; une ligne par banque × date × code, valeur retenue (révisée par le document suivant
// quand elle existe). Sert au ROE et au ROA (spécification 7.7).
let
    Fichier = Fichiers_Donnees{[fichier = "etats_financiers_versions.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"date_arrete", type date}, {"type", type text}, {"code", type text},
            {"version", type text}, {"valeur_mdt", type number}, {"fichier", type text}, {"page",
Int64.Type},
            {"statut_valeur", type text}, {"valeur_retenue", type number}, {"source_valeur", type text},
            {"statut_revision", type text}, {"ecart_mdt", type number}, {"remarque", type text}
        },
        "en-US"
    ),
    VidesEnNull = Table.ReplaceValue(
        Types, "", null, Replacer.ReplaceValue, {"statut_valeur", "statut_revision", "remarque"}
    ),
    // filtre explicite : seule la version retenue entre dans les faits (même règle que les indicateurs)
    VersionRetenue = Table.SelectRows(VidesEnNull, each [version] = [source_valeur]),
    Colonnes = Table.SelectColumns(
        VersionRetenue,
        {"banque", "date_arrete", "type", "code", "valeur_retenue", "version", "fichier", "page",
         "statut_valeur", "statut_revision", "remarque"}
    ),
    Renommees = Table.RenameColumns(Colonnes, {{"valeur_retenue", "valeur_mdt"}}),
    // colonne conditionnelle : annualisation du résultat net (un semestre × 2, un exercice × 1)
    Annualisation = Table.AddColumn(
        Renommees, "facteur_annualisation", each if [type] = "semestriel" then 2 else 1, Int64.Type
    ),
    // segment d'actionnariat : catégorie de la banque à la date d'arrêté, selon les périodes de
    // Banques_Actionnariat (fonction Actionnariat_A_La_Date ; UBCI : groupe étranger jusqu'au 15/03/2021)
    Periodes = Table.Buffer(
        Table.SelectColumns(Banques_Actionnariat, {"code_banque", "date_debut", "date_fin", "actionnariat"})
    ),
    Segment = Table.AddColumn(
        Annualisation, "Segment_Actionnariat", each Actionnariat_A_La_Date(Periodes, [banque],
[date_arrete]), type text
    )
in
    Segment
```

Revisions_Etats_Financiers

```
// Auteur : Ahmed Ben Arfa
// Versions écartées des états financiers (chargée) : chaque valeur publiée puis remplacée par la colonne
// N-1 du document suivant de même type (ou révisée non retenue : BNA 30/06/2025), avec l'écart à la valeur
// retenue. Page « qualité des données » (spécification 7.7). Complément exact de Fait_Etats_Financiers :
// même fichier, filtre inverse.
let
    Fichier = Fichiers_Donnees{[fichier = "etats_financiers_versions.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"date_arrete", type date}, {"type", type text}, {"code", type text},
            {"version", type text}, {"valeur_mdt", type number}, {"fichier", type text}, {"page",
Int64.Type},
            {"statut_valeur", type text}, {"valeur_retenue", type number}, {"source_valeur", type text},
            {"statut_revision", type text}, {"ecart_mdt", type number}, {"remarque", type text}
        },
        "en-US"
    ),
    VidesEnNull = Table.ReplaceValue(
        Types, "", null, Replacer.ReplaceValue, {"statut_valeur", "statut_revision", "remarque"}
    ),
    // filtre explicite, inverse de celui de Fait_Etats_Financiers : toutes les versions écartées
    VersionsNonRetenues = Table.SelectRows(VidesEnNull, each [version] <> [source_valeur]),
    Colonnes = Table.SelectColumns(
        VersionsNonRetenues,
        {"banque", "date_arrete", "type", "code", "version", "valeur_mdt", "fichier", "page",
         "valeur_retenue", "source_valeur", "statut_revision", "remarque"}
    ),
    // écart calculé ici (la colonne ecart_mdt du CSV n'est pas reprise) : de combien la valeur retenue
    // s'écarte de cette version
    EcartMDT = Table.AddColumn(Colonnes, "Ecart_MDT", each [valeur_retenue] - [valeur_mdt], type nullable
number),
    EcartPct = Table.AddColumn(
        EcartMDT, "Ecart_Pct",
        each if [valeur_mdt] = null or [valeur_mdt] = 0 then null else [Ecart_MDT] /
Number.Abs([valeur_mdt]),
        type nullable number
    ),
    // colonne conditionnelle : statut de la révision en clair, pour la page qualité
    TypeRevision = Table.AddColumn(
        EcartPct, "Type_Revision",
        each if [statut_revision] = null then "identique à la valeur retenue (arrondi)"
            else if [statut_revision] = "retraitement_mineur" then "retraitement mineur (2 % au plus)"
            else if [statut_revision] = "revision_importante" then "révision importante (plus de 2 %)"
            else if Text.StartsWith([statut_revision], "revisee non retenue") then "révisée non retenue
(modification comptable incluse)"
            else [statut_revision],
        type text
    )
in
    TypeRevision
```

Controle

```
// Auteur : Ahmed Ben Arfa
// Contrôle de l'extraction (chargée) : une ligne par document trimestriel (12 banques × 28 trimestres),
// avec son statut, le nombre de valeurs lues, les alertes, le fichier et le gabarit de mise en page.
// Page « qualité des données » (spécification 5.1 et 7.8).
let
    Fichier = Fichiers_Donnees{[fichier = "controle_extraction.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"trimestre", type text}, {"statut", type text}, {"nb_valeurs",
Int64.Type},
            {"alertes", type text}, {"fichier", type text}, {"gabarit", type text}
        },
        "en-US"
    ),
    VidesEnNull = Table.ReplaceValue(Types, "", null, Replacer.ReplaceValue, {"alertes", "gabarit"}),
    // date d'arrêté calculée à partir du trimestre (« 2024T3 » → 30/09/2024), pour la relier aux dates
    DateArrete = Table.AddColumn(
        VidesEnNull, "date_arrete",
        each Date.EndOfMonth(#date(Number.From(Text.Start([trimestre], 4)), 3 *
Number.From(Text.End([trimestre], 1)), 1)),
        type date
    ),
    // nombre d'alertes : elles sont séparées par « | » dans le fichier
    NbAlertes = Table.AddColumn(
        DateArrete, "nb_alertes",
        each if [alertes] = null then 0 else List.Count(Text.Split([alertes], " | ")),
        Int64.Type
    ),
    // colonne conditionnelle : statut en clair, regroupé pour la page qualité
    CategorieStatut = Table.AddColumn(
        NbAlertes, "Categorie_Statut",
        each if [statut] = "ok" then "conforme"
            else if List.Contains({"retraitement possible", "ecart trimestre/cumul", "ecart composantes"},
[statut])
                then "écart signalé (vient de la banque, non bloquant)"
            else if [statut] = "saisie manuelle" then "tableau en image, saisi à la main"
            else if [statut] = "illisible (image)" then "tableau en image, repris du document suivant"
            else if [statut] = "a verifier" then "alerte bloquante (erreur du document vérifiée)"
            else [statut],
        type text
    )
in
    CategorieStatut
```

Ecarts_Annuels

```
// Auteur : Ahmed Ben Arfa
// Requête intermédiaire (non chargée) : cause de chacun des écarts du contrôle annuel (somme des 4
// trimestres retenus ≠ cumul annuel retenu), examinés un à un dans les PDF : catégorie (retraitement de
// la banque, erreur dans le document de la banque, reclassement, arrondi, erreur d'extraction), cause,
// fichier et page de la preuve, correction faite. Fichier versionné docs\ecarts_annuels.csv, lu dans le
// dossier du projet. Controle_Annuel en prend la cause par une fusion.
let
    Source = Csv.Document(
        File.Contents(CheminDonnees & "\docs\ecarts_annuels.csv"),
        [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]
    ),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"annee", Int64.Type}, {"code_indicateur", type text},
            {"somme_trimestres", type number}, {"cumul_T4", type number}, {"ecart_mdt", type number},
            {"categorie", type text}, {"cause", type text}, {"fichier", type text}, {"page", Int64.Type},
            {"correction", type text}
        },
        "en-US"
    ),
    // seules la clé et l'explication servent à la fusion (les montants sont recalculés dans Controle_Annuel)
    Colonnes = Table.SelectColumns(
        Types, {"banque", "annee", "code_indicateur", "categorie", "cause", "fichier", "page", "correction"}
    )
in
    Colonnes
```

Controle_Annuel

```
// Auteur : Ahmed Ben Arfa
// Contrôle annuel (chargée) : pour chaque banque, année (2020–2025) et indicateur de flux, somme des 4
// trimestres retenus comparée au cumul annuel retenu, avec le statut et, pour chaque écart, sa cause
// (Ecarts_Annuels). Page « qualité des données » (spécification 5.1 et 5.2).
let
    Fichier = Fichiers_Donnees{[fichier = "controle_annuel.csv"]}[contenu],
    Source = Csv.Document(Fichier, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Types = Table.TransformColumnTypes(
        EnTetes,
        {
            {"banque", type text}, {"annee", Int64.Type}, {"code_indicateur", type text},
            {"somme_trimestres", type number}, {"cumul_T4", type number}, {"source_cumul", type text},
            {"ecart", type number}, {"statut", type text}
        },
        "en-US"
    ),
    // écart recalculé ici (la colonne du CSV n'est pas reprise) : somme des trimestres – cumul annuel
    Ecart = Table.AddColumn(
        Table.RemoveColumns(Types, {"ecart"}), "Ecart_MDT", each [somme_trimestres] - [cumul_T4], type number
    ),
    // date de l'exercice (31/12), pour la relier aux dates
    DateArrete = Table.AddColumn(Ecart, "date_arrete", each #date([annee], 12, 31), type date),
    // fusion avec Ecarts_Annuels : la cause de chaque écart (vide pour les autres statuts)
    Fusion = Table.NestedJoin(
        DateArrete, {"banque", "annee", "code_indicateur"},
        Ecarts_Annuels, {"banque", "annee", "code_indicateur"}, "Cause", JoinKind.LeftOuter
    ),
    Developpee = Table.ExpandTableColumn(
        Fusion, "Cause", {"categorie", "cause", "fichier", "page", "correction"},
        {"categorie_ecart", "cause_ecart", "fichier_preuve", "page_preuve", "correction"}
    ),
    // colonne conditionnelle : statut en clair pour la page qualité
    StatutClair = Table.AddColumn(
        Developpee, "Statut_Clair",
        each if [statut] = "ok" then "somme des trimestres = cumul annuel"
            else if [statut] = "en attente de révision" then "en attente du document de l'année suivante"
            else if [statut] = "rupture de série" then "changement de méthode en cours d'année"
            else if [statut] = "ecart" and [categorie_ecart] <> null then "écart expliqué : " &
[categorie_ecart]
            else if [statut] = "ecart" then "écart sans cause écrite"
            else [statut],
        type text
    )
in
    StatutClair
```

Gabarits

```
// Auteur : Ahmed Ben Arfa
// Requête intermédiaire (non chargée) : gabarits de mise en page des documents de chaque banque
// (gabarits.csv, versionné à la racine, lu dans le dossier du projet) : période, colonnes, unité, et
// définitions des charges opératoires et des capitaux propres trimestriels de la banque.
// Definitions_Charges et Definitions_Capitaux_Propres en partent.
let
    // 11 colonnes : la remarque (dernière colonne) contient parfois un « ; », le reste est ignoré
    Source = Csv.Document(
        File.Contents(CheminDonnees & "\gabarits.csv"),
        [Delimiter = ";", Columns = 11, Encoding = 65001, QuoteStyle = QuoteStyle.Csv]
    ),
    EnTetes = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Colonnes = Table.SelectColumns(
        EnTetes,
        {"banque", "premier_trimestre", "dernier_trimestre", "definition_charges_operatoires",
         "definition_capitaux_propres"}
    ),
    Types = Table.TransformColumnTypes(
        Colonnes,
        {{"banque", type text}, {"premier_trimestre", type text}, {"dernier_trimestre", type text},
         {"definition_charges_operatoires", type text}, {"definition_capitaux_propres", type text}}
    )
in
    Types
```

Definitions_Charges

```
// Auteur : Ahmed Ben Arfa
// Contenu des charges opératoires publiées par chaque banque, par période (chargée) : sans dotations,
// avec dotations publiées, ou avec un reste non détaillé. Explique pourquoi le coefficient publié n'est
// pas comparable d'une banque à l'autre (info-bulle du coefficient publié, page qualité ; spécification
// 7.2 et 7.4). Les classements utilisent le coefficient harmonisé.
let
    Source = Gabarits,
    // regroupement : les gabarits découpent l'historique par mise en page (et par rang du trimestre) ;
    // une définition garde la période allant de son premier à son dernier trimestre
    Periodes = Table.Group(
        Source, {"banque", "definition_charges_operatoires"},
        {{"premier_trimestre", each List.Min([premier_trimestre]), type text},
         {"dernier_trimestre", each List.Max([dernier_trimestre]), type text}}
    ),
    // colonne conditionnelle : la définition en clair (mêmes textes que le script d'extraction)
    Description = Table.AddColumn(
        Periodes, "description",
        each if [definition_charges_operatoires] = "sans dotations"
                then "total publié = frais de personnel + charges générales d'exploitation"
            else if [definition_charges_operatoires] = "avec dotations publiees"
                then "total publié = frais de personnel + charges générales + dotations aux amortissements,
les trois lignes publiées"
            else if [definition_charges_operatoires] = "avec reste non detaille"
                then "total publié > frais de personnel + charges générales ; la différence (dotations ou
autres charges) n'est pas détaillée dans le PDF"
            else if [definition_charges_operatoires] = "composantes non publiees"
                then "seul le total est publié ; coefficient harmonisé non calculable"
            else null,
        type text
    ),
    Triees = Table.Sort(Description, {{"banque", Order.Ascending}, {"premier_trimestre", Order.Ascending}})
in
    Triees
```

Definitions_Capitaux_Propres

```
// Auteur : Ahmed Ben Arfa
// Contenu des capitaux propres trimestriels publiés par chaque banque (chargée) : hors résultat de la
// période, résultat inclus, ou hors résultat avec affectation au 31/12. Affiché dans la fiche banque à côté
// des capitaux propres trimestriels ; les comparaisons entre banques utilisent les capitaux propres du
// bilan (spécification 5.2 et 7.7, décision du 19/10/2026).
let
    Source = Gabarits,
    Colonnes = Table.SelectColumns(Source, {"banque", "definition_capitaux_propres"}),
    // une ligne par banque (la définition ne change pas d'un gabarit à l'autre)
    Distinctes = Table.Distinct(Colonnes),
    // colonne conditionnelle : la définition en clair (mêmes textes que le script d'extraction)
    Description = Table.AddColumn(
        Distinctes, "description",
        each if [definition_capitaux_propres] = "hors resultat de la periode"
                then "capitaux propres sans le résultat de la période en cours (= capitaux propres du bilan –
résultat net de la période)"
            else if [definition_capitaux_propres] = "resultat inclus"
                then "capitaux propres y compris le résultat de la période (= capitaux propres du bilan)"
            else if [definition_capitaux_propres] = "hors resultat, apres affectation au 31/12"
                then "hors résultat de la période en cours d'année ; au 31/12, après affectation du résultat
de l'exercice (réserves augmentées, dividendes déduits)"
            else null,
        type text
    ),
    Triees = Table.Sort(Description, {{"banque", Order.Ascending}})
in
    Triees
```

Annexe A bis — Code DAX

Texte exact des fichiers `dax/*.dax`, identique au code du projet `.pbip`.

Dim_Date

```
// Auteur : Ahmed Ben Arfa
// Table des dates (spécification 6) : une ligne par jour du 01/01/2019 au 31/12/2026.
// Annee_Trimestre a le format des faits (« 2024T3 ») et se trie par Cle_Trimestre (20243) ;
// Periode : Avant Covid avant le 01/03/2020, Covid jusqu'au 31/12/2021, Après Covid ensuite,
// triée par Ordre_Periode (sinon l'ordre alphabétique mettrait « Après Covid » en premier).
ADDCOLUMNS (
    CALENDAR ( DATE ( 2019, 1, 1 ), DATE ( 2026, 12, 31 ) ),
    "Annee", YEAR ( [Date] ),
    "Trimestre", "T" & QUARTER ( [Date] ),
    "Annee_Trimestre", YEAR ( [Date] ) & "T" & QUARTER ( [Date] ),
    "Cle_Trimestre", YEAR ( [Date] ) * 10 + QUARTER ( [Date] ),
    "Fin_Trimestre", [Date] = EOMONTH ( [Date], 0 ) && MOD ( MONTH ( [Date] ), 3 ) = 0,
    "Periode",
        SWITCH (
            TRUE (),
            [Date] < DATE ( 2020, 3, 1 ), "Avant Covid",
            [Date] < DATE ( 2022, 1, 1 ), "Covid",
            "Après Covid"
        ),
    "Ordre_Periode",
        SWITCH (
            TRUE (),
            [Date] < DATE ( 2020, 3, 1 ), 1,
            [Date] < DATE ( 2022, 1, 1 ), 2,
            3
        )
)
```

Annexe B — Feuille de route et décisions

État d'avancement du projet et décisions prises, tenus à jour après chaque étape terminée (chaque requête Power Query, chaque groupe de mesures, chaque décision). La présentation du projet est dans le README, le détail des règles dans la spécification.

Dernière mise à jour : 23/10/2026.

État d'avancement

#	Étape	État
1	Script de collecte et d'extraction des indicateurs trimestriels	Fait
2	Collecte réelle T4 2019 → T3 2026 et corrections	Fait : 336/336 documents, 12 gabarits de mise en page ; 13 documents publiés en image saisis à la main ; 11 545 valeurs retenues
3	Validation sur l'image des PDF	Valeurs de référence relevées à la main (438 sur 41 documents) ; `docs/validation.md` à écrire
4	Cours boursiers 2019–2026	Fait : 1 955 séances par banque, 63 opérations sur titres sourcées
4b	États financiers (ROE, ROA)	Fait : 168 arrêtés, 504 valeurs retenues
4c	Écarts du contrôle annuel	Fait : 22 écarts, cause et source de chacun (`docs/ecarts_annuels.csv`)
5	Requêtes Power Query	**Terminée le 23/10/2026** : 22 requêtes, toutes vérifiées dans Power BI Desktop (détail ci-dessous)
6	Modèle en étoile et mesures DAX	**En cours** : table des dates vérifiée ; 28 relations écrites, à vérifier dans Power BI Desktop ; ensuite mesures
7	Pages du rapport	À faire
8	README final et captures	README réécrit, section « Ouvrir le rapport » ; captures et lien de publication sur le web à faire

Étape 5 : requêtes Power Query (terminée)

Rôle	Requêtes (nombre de lignes)
Paramètres	`CheminDonnees` (dossier du projet), `AdresseDepot` (fichiers bruts du dépôt GitHub)
Intermédiaires, non chargées	`Fichiers_Donnees` , `Versions` , `Operations_Titres` , `Ecarts_Annuels` , `Gabarits` ; fonction personnalisée `Actionnariat_A_La_Date`
Dimensions	`Dim_Indicateur` (27), `Dim_Banque` (12), `Banques_Actionnariat` (15)
Faits	`Fait_Indicateurs` (7 173), `Variations_Comparables` (10 981), `Fait_Etats_Financiers` (504), `Fait_Cours` (23 460), `Evenements` (11)
Qualité et définitions	`Revisions` (14 884), `Revisions_Etats_Financiers` (423), `Controle` (336), `Controle_Annuel` (288), `Definitions_Charges` (13), `Definitions_Capitaux_Propres` (12)

Trois types de sources (dossier de CSV, fichier Excel, adresse GitHub) ; dépivotage et pivot, fusions, types avec paramètres régionaux, colonnes conditionnelles, regroupements, fonction personnalisée (segment d'actionnariat à la date de chaque valeur). Chaque étape est expliquée dans `guide_powerbi.md` et dans `Guide_PowerBI.pdf` .

Étape 6 (en cours) : modèle et mesures DAX

Un groupe à la fois, vérifié dans Power BI Desktop avant le suivant :

1. **Table des dates** (faite le 23/10/2026, `dax/Dim_Date.dax`, vérifiée dans Power BI Desktop ; attendu : 2 922 jours, 32 fins de trimestre, Avant Covid 425 / Covid 671 / Après Covid 1 826 jours) : du 01/01/2019 au 31/12/2026, année, trimestre, libellé du trimestre trié chronologiquement, fin de trimestre, **période Covid** (avant le 01/03/2020, jusqu'au 31/12/2021, après) triée par une colonne d'ordre ; libellé du trimestre au format des faits (« 2024T3 ») ; marquée comme table de dates.
2. **Relations du modèle en étoile** (28 relations écrites le 23/10/2026, à vérifier ; clés contrôlées : aucune banque, aucun code, aucune date des faits absents de leur dimension) (une dimension vers plusieurs lignes, filtre simple) : dates, banques et indicateurs vers les tables de faits, de qualité et d'événements.
3. **Mesures DAX par groupes** : bases ; totaux du secteur, nombre de banques présentes et dernier trimestre complet ; ratios ; croissances ; ROE et ROA ; Bourse ; qualité des données.

Ensuite : pages du rapport (étape 7), `docs/validation.md` (étape 3), captures et publication sur le web avec le lien en tête du README (étape 8).

Règles pour les mesures DAX

- Totaux du secteur, parts de marché, rangs et indice de concentration **seulement quand les 12 banques sont présentes** ; sinon vides ou marqués « incomplet », avec le nombre de banques présentes affiché.
- Filtre par défaut sur le **dernier trimestre complet** (12 banques présentes), pas sur le dernier trimestre qui contient une donnée.
- **Croissances sur un an** calculées avec `Variations_Comparables` (valeur N et N-1 du même document), jamais en rapprochant deux documents.
- **Chiffres annuels** pris dans le cumul de l'année, jamais en additionnant les quatre trimestres.
- Soldes (crédits, dépôts, capitaux propres) pris à la date, jamais additionnés dans le temps.
- **Coefficient d'exploitation harmonisé** (frais de personnel + charges générales) / PNB pour toutes les comparaisons ; le coefficient publié reste une information, avec la définition de chaque banque.
- **Aucune valeur absente remplacée par zéro.**
- **ROE et ROA** avec les capitaux propres et le total de l'actif du bilan (états financiers), le résultat net avant modifications comptables annualisé, et la moyenne entre l'ouverture de l'exercice et la date ; les capitaux propres trimestriels ne servent qu'à la fiche banque.

Trimestres incomplets pour le secteur

Vérifié le 23/10/2026 sur les valeurs retenues (28 trimestres, de 2019T4 à 2026T3) :

- **complets (28/28)** : PNB, produits et charges d'exploitation bancaire, charges opératoires, frais de personnel, charges générales (trimestre seul et cumul), crédits, dépôts ;
- **complets (28/28) depuis le 23/10/2026** : intérêts perçus et encourus, commissions en produits et encourues, revenus du portefeuille (donc marge d'intérêts et marge sur commissions) ;
- **22/28** : portefeuille-titres, dépôts à vue et d'épargne, emprunts : il manque BTE de 2020T1 à 2020T4, 2021T1 et 2021T3 (documents publiés en image, détail du bilan non saisi) ;
- capitaux propres trimestriels : 20/28 (BTE), affichés seulement dans la fiche banque ;
- jamais complets, publiés par une partie des banques seulement : dotations aux amortissements, dépôts à terme, autres dépôts, portefeuille commercial, autres produits d'exploitation (hors indicateurs du rapport).

Le total du secteur d'un indicateur n'est calculé que pour ses trimestres complets.

Décisions prises

Date	Sujet	Décision
17/10/2026	Révisions des banques	La valeur retenue est toujours la version révisée quand elle existe (colonne N-1 du document de l'année suivante) ; écart mesuré sur le cumul ; plus de 2 % = révision importante et rupture de série
17/10/2026	Colonnes de comparaison fausses	Une colonne N-1 fausse dans le document d'une banque n'est jamais retenue (UIB 2023T2 par règle automatique, Wifak 2023T4 par `exclusions_colonnes.csv`)
17/10/2026	Coefficient d'exploitation	Coefficient harmonisé (frais de personnel + charges générales) / PNB pour toutes les comparaisons ; coefficient publié affiché pour information
17/10/2026	Documents en image	BH 2019T4 repris du document suivant, sans saisie ; BTE saisi à la main, avec le fichier, la page et la vérification de chaque valeur
18/10/2026	ROE et ROA	Résultat net avant modifications comptables, pour toutes les banques
18/10/2026	ilboursa	Comparaison du résultat net purement informative
19/10/2026	Capitaux propres	Comparaisons entre banques avec les capitaux propres du bilan ; capitaux propres trimestriels dans la fiche banque seulement, avec la définition de chaque banque
19/10/2026	États financiers manquants	Recherche dans l'ordre : site de la BVMT, liste du CMF, Bulletins Officiels ; même règle des versions que les indicateurs
20/10/2026	BNA S1 2025	Le résultat avant modifications comptables prime sur la règle des versions
20/10/2026	Nombre mal formé (Attijari S1 2026)	Déduit seulement s'il se déduit exactement d'un total du même document
20/10/2026	Cours 2026	Bulletins quotidiens de la Bourse pour les cours, volume d'ilboursa ; ajustements calculés dans Power Query à partir de `operations_titres.csv` ; 10 s entre deux requêtes vers la Bourse
21/10/2026	Ouvertures 2022–2025	Pas de téléchargement des bulletins quotidiens : ouvertures laissées vides
22/10/2026	BTE	Saisie des 13 documents publiés en image (indicateurs principaux)
22/10/2026	Actionnariat	Classement par le contrôle, en 4 catégories (spécification 5.3)
23/10/2026	Publication	Dépôt public ; adresse e-mail de l'auteur gardée dans l'historique ; audit avant chaque push ; code sous licence MIT
23/10/2026	Données pour Power BI	CSV finaux versionnés dans `donnees_publiees/`, régénérés par `scripts/publier_donnees.py`
23/10/2026	Projet Power BI	Enregistré en projet `.pbip` (modèle en texte TMDL, versionné) ; cache des données et réglages du poste non versionnés
23/10/2026	Valeurs vides de `Fait_Indicateurs`	Vides gardés (jamais zéro) ; `Valeur_Trim` vidé pour les stocks par une colonne conditionnelle après fusion avec `Dim_Indicateur` ; « non » pour les ruptures absentes
23/10/2026	BTE T4 2021 et T4 2022	Saisie des composantes du trimestre seul (intérêts, commissions, portefeuille, autres produits) avec les versions révisées des documents suivants ; autres produits de 2023T4 vérifiés sur l'image (0)
23/10/2026	BTE 2020T1 à 2021T3	Saisie des composantes (intérêts, commissions, portefeuille, autres produits) de 2020T1–T4, 2021T1 et 2021T3, avec les colonnes N-1 des documents suivants : 216 valeurs ; intérêts, commissions et portefeuille complets pour le secteur à 28/28 trimestres
23/10/2026	Documentation des transformations	`docs/Guide_PowerBI.pdf` : chaque transformation expliquée, code M de chaque requête et feuille de route en annexe, régénéré après chaque requête ou groupe de mesures
23/10/2026	Niveaux de confidentialité	Même niveau pour toutes les sources (Professionnel), dossier du projet plutôt que tout le disque, accès anonyme à GitHub
23/10/2026	Facteur d'ajustement « prix »	Dividende détaché le même jour qu'une attribution retiré de la clôture de la veille avant la parité (BT 10/05/2022 : 0,834 au lieu de 0,796)

Date	Sujet	Décision
23/10/2026	BTE, actionnariat 2024	Date de l'augmentation de capital introuvable dans les états financiers : seconde période à partir du 01/07/2024, « date exacte inconnue, entre le 30/06 et le 31/12/2024 » ; périodes continues contrôlées par un test
23/10/2026	Ouverture du projet par un tiers	Cache des données Power BI versionné (données publiques) : le projet s'ouvre avec ses données sans réglage ; section « Ouvrir le rapport » du README ; publication sur le web à la fin
23/10/2026	Documentation des fonctions M	Tableau de chaque fonction M utilisée (ce qu'elle fait, requêtes) dans le guide et son PDF, contrôlé par un test
23/10/2026	Étape 5 terminée	Les 22 requêtes Power Query sont vérifiées ; passage à l'étape 6 (table des dates, relations, mesures DAX)
23/10/2026	Relations	28 relations 1 → plusieurs, filtre simple : dates, banques, indicateurs vers les faits, la qualité et les événements ; `Fait_Cours` relié par le code de la banque (pas le ticker) ; actionnariat et définitions reliés aux banques
23/10/2026	Table des dates	Code de la spécification, libellé du trimestre « 2024T3 » (format des faits) au lieu de « 2024-T3 », période Covid triée par `Ordre_Periode` ; tables calculées écrites par `scripts/ecrire_tmdl_table.py` (outil existant étendu) ; code DAX dans `dax/`, en annexe du guide en PDF